

A Comprehensive Survey on Coded Distributed Computing: Fundamentals, Challenges, and Networking Applications

Jer Shyuan Ng¹, Graduate Student Member, IEEE, Wei Yang Bryan Lim², Nguyen Cong Luong³,
Zehui Xiong⁴, Member, IEEE, Alia Asheralieva⁵, Dusit Niyato⁶, Fellow, IEEE,
Cyril Leung⁷, Life Member, IEEE, and Chunyan Miao⁸, Senior Member, IEEE

Abstract—Distributed computing has become a common approach for large-scale computation tasks due to benefits such as high reliability, scalability, computation speed, and cost-effectiveness. However, distributed computing faces critical issues related to communication load and straggler effects. In particular, computing nodes need to exchange intermediate results with each other in order to calculate the final result, and this significantly increases communication overheads. Furthermore, a distributed computing network may include straggling nodes that run intermittently slower. This results in a longer overall time needed to execute the computation tasks, thereby limiting the performance of distributed computing. To address these issues, coded distributed computing (CDC), i.e., a combination of coding theoretic techniques and distributed computing, has been recently proposed as a promising solution. Coding theoretic techniques have proved effective in WiFi and cellular systems to deal with channel noise. Therefore, CDC may significantly reduce communication load, alleviate the effects of stragglers, provide

fault-tolerance, privacy and security. In this survey, we first introduce the fundamentals of CDC, followed by basic CDC schemes. Then, we review and analyze a number of CDC approaches proposed to reduce the communication costs, mitigate the straggler effects, and guarantee privacy and security. Furthermore, we present and discuss applications of CDC in modern computer networks. Finally, we highlight important challenges and promising research directions related to CDC.

Index Terms—Distributed computing, communication minimization, straggler effects mitigation, security, coded distributed computing.

I. INTRODUCTION

WITH the generation of continuous data streams, a single computing node is no longer capable of handling the large-scale computation tasks individually. In recent years, distributed computing has been used for large-scale computations [1] since it offers several advantages over centralized computing. Distributed computing refers to a group of computing nodes that work together to solve a computation task as a single system by sharing their networking and storage resources. First, distributed computing is able to provide computing services with high reliability and fault-tolerance. In particular, distributed computing systems can efficiently and reliably work even if some of the computing nodes, i.e., computers or workers, fail. Second, distributed computing has high computation speed as the computation load is shared among various computing nodes. Third, distributed computing systems are scalable since computing nodes can easily be added. Fourth, distributed computing is economical by using computing nodes with low-cost hardware. Likewise, distributed computing is adopted in cloud computing and other emerging services. Given the aforementioned advantages, distributed computing has been applied in numerous real-life applications such as telecommunication networks [2] (e.g., telephone networks and wireless sensor networks), network applications [3] (e.g., world-wide Web networks, massively multiplayer online games and virtual reality communities, distributed database management systems, and network file systems), real-time process control [4] (e.g., aircraft control systems), and parallel computation [5] (e.g., cluster computing, grid computing, and computer graphics).

Manuscript received August 20, 2020; revised January 19, 2021; accepted May 31, 2021. Date of publication June 23, 2021; date of current version August 23, 2021. This work was supported in part by the Alibaba Group through Alibaba Innovative Research (AIR) Program and Alibaba-NTU Singapore Joint Research Institute (JRI); in part by Singapore Energy Market Authority (EMA), Energy Resilience, under Grant NRF2017EWTEP003-04; in part by WASP/NTU under Grant M4082187 (4080); in part by the Singapore Ministry of Education (MOE) Tier 1 (RG16/20); and in part by SUTD under Grant SRG-ISTD-2021-165. (Corresponding author: Nguyen Cong Luong.)

Jer Shyuan Ng and Wei Yang Bryan Lim are with Alibaba Group and Alibaba-NTU Joint Research Institute, Nanyang Technological University, Singapore (e-mail: s190068@e.ntu.edu.sg; limw0201@e.ntu.edu.sg).

Nguyen Cong Luong is with the Faculty of Computer Science and PHENIKAA Research and Technology Institute, PHENIKAA University, Hanoi 12116, Vietnam (e-mail: luong.nguyencong@phenikaa-uni.edu.vn).

Zehui Xiong is with the Pillar of Information Systems Technology and Design, Singapore University of Technology Design, Singapore (e-mail: zehui_xiong@sutd.edu.sg).

Alia Asheralieva is with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: aasheralieva@gmail.com).

Dusit Niyato is with the School of Computer Science and Engineering, Nanyang Technological University, Singapore (e-mail: dniyato@ntu.edu.sg).

Cyril Leung is with the Department of Electrical and Computer Engineering and Joint NTU-UBC Research Centre of Excellence in Active Living for the Elderly (LILY), The University of British Columbia, Vancouver, BC V6T 1Z4, Canada (e-mail: cleung@ece.ubc.ca).

Chunyan Miao is with the Joint NTU-UBC Research Centre of Excellence in Active Living for the Elderly (LILY) and the School of Computer Science and Engineering, Nanyang Technological University, Singapore (e-mail: ascymiao@ntu.edu.sg).

Digital Object Identifier 10.1109/COMST.2021.3091684

However, distributed computing faces serious challenges. Let us consider one of the most common distributed computation frameworks, i.e., MapReduce [6]. MapReduce is a software framework and programming model for processing a computation task across large datasets using a large number of computing nodes. A set of the computing nodes is referred to as a cluster or a grid. In general, the overall computation task is decomposed into three phases, i.e., the “Map” phase, “Shuffle” phase, and “Reduce” phase. In the Map phase, a master splits the computation task into multiple subtasks and assigns the subtasks to the computing nodes. The computing nodes compute the subtasks according to the allocated Map functions to generate intermediate results. Then, the intermediate results are exchanged among the computing nodes, namely “data shuffling”, during the Shuffle phase. In the Reduce phase, the computing nodes use these results to compute the final result in a distributed manner by using their allocated Reduce functions.

Distributed computing has two major challenges. First, the computing nodes need to exchange a number of intermediate results over the network with each other in order to calculate the final result; this significantly increases communication overheads and limits the performance of distributed computing applications such as Self-Join [7], Terasort [8], and machine learning [9]. For example, for the Hadoop cluster at Facebook, it is observed that on average, the data shuffling phase accounts for 33% of the overall job execution time [9]. 65% and 70% of the overall job execution time is spent on the Shuffle phase when running a TeraSort and Self-Join application respectively on a heterogeneous Amazon EC2 cluster [10]. In fact, the communication bottleneck is worse in the trainings of convolutional neural networks (CNNs), e.g., Resnet-50 [11] and AlexNet [12], which include updates of millions of model parameters. Second, distributed computing is executed by a large number of computing nodes which may have very different computing and networking resources. As a result, there are straggling nodes or stragglers, i.e., computing nodes which run unintentionally slower than others, thereby increasing the overall time needed to complete the computing tasks. To address the straggler effects, traditional approaches such as work exchange [13] and naive replication [14] have been adopted for the distributed computing. However, such approaches either introduce redundancy or require coordination among the nodes that significantly increases communication costs and computation load. This motivates the need for a novel technique that is able to more effectively and completely address the straggler effects and communication load of distributed computing.

Coding theoretic techniques, e.g., channel coding such as low-density parity-check (LDPC) [15], have been widely used in WiFi and cellular systems to combat the impact of channel noise and impairments. They have also been applied in distributed storage systems and cache networks [16] to reduce storage cost and network traffic. The basic principle of the coding theoretic techniques is that redundant information, i.e., redundancy, is introduced in messages/signals before they are transmitted to a receiver. The redundancy is included in the messages in a controller manner such that it can be

utilized by the receiver to correct errors caused by the channel noise. Coding theoretic techniques have been recently regarded as promising solutions to cope with the challenges in distributed computing [17], [18]. For example, coding theoretic techniques can be used to encode the Map tasks of the computing nodes such that the master is able to recover the final result from partially finished nodes, thus alleviating the straggler effects [19]. Another example is that coding theoretic techniques enable coding opportunities across intermediate results of the distributed computation tasks which significantly reduces the communication load by reducing the number and the size of data transmissions among the computing nodes [17]. The combination of coding techniques and distributed computing is called coded distributed computing (CDC) [17]. Apart from reducing communication load and alleviating the effects of stragglers, CDC can provide fault-tolerance, preserve privacy [20], and improve security [21] in distributed computing. As a result, CDC approaches have recently received a lot of attention.

With the surge of data generated, Artificial Intelligence (AI) and Internet of Things (IoT) applications are developed for many smart city applications such as the estimation of road conditions [22], tracking of medical conditions [23] and air quality monitoring [24], thereby transforming and improving various aspects of our life. With CDC, Artificial Intelligence (AI) and IoT applications that require the processing of large-scale datasets for analysis and predictions can be implemented efficiently and securely. Besides, CDC schemes can be applied in modern networks such as Network Function Virtualization (NFV) and edge computing. With the data mainly generated by end devices, e.g., IoT, that have significant sensing as well as computational and storage capabilities, it is natural to perform some computations at the end devices, instead of the cloud which may not be able to handle the large amounts of data generated. As such, edge computing [25] has been proposed as a solution to perform distributed computation tasks. In order to perform complex computations, e.g., the training of deep neural networks that involve a large number of training layers, resource-constrained devices may need to pool their resources to perform their computations collaboratively [26]. This results in high communication costs and computation latency. CDC schemes can be implemented to overcome these challenges. Furthermore, CDC schemes can be implemented on edge computing networks that involve constantly-moving end devices [27], e.g., vehicles and smartphones, which imposes additional communication constraints.

To the best of our knowledge, although there are several surveys and books related to distributed computing, there is no survey paper on CDC. In particular, large-scale distributed computing and applications are discussed in [1]. Surveys related to distributed computing include grid resource management systems for distributed computing [28], resource allocation in high performance distributed computing [29], wireless distributed computing [30], and wireless grid computing [31]. This motivates the need for this survey on CDC. In summary, our survey has the following contributions.

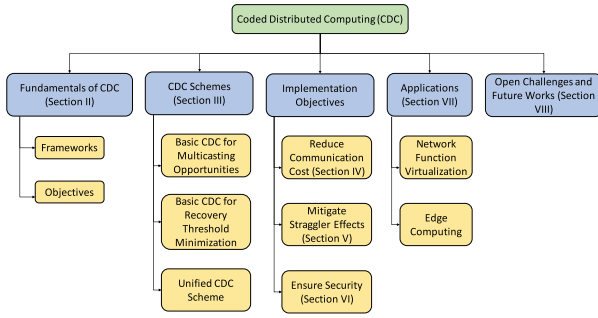


Fig. 1. Structure of survey.

- We describe the fundamentals of CDC. In particular, we introduce the commonly used distributed computation frameworks for the implementation of coding techniques and algorithms. We then discuss basic CDC schemes.
- We review and discuss a number of CDC schemes to reduce the communication costs for distributed computing. The approaches include file allocation, coded shuffling design, and function allocation. We further analyze and compare the advantages and disadvantages of the CDC schemes.
- We review, discuss, and analyze CDC schemes which mitigate the straggler effects of distributed computing. The approaches include computation load allocations, approximate coding, and exploitation of stragglers.
- We review and present CDC schemes that can improve the privacy and security in distributed computing.
- We analyze and provide insights into the existing approaches and solutions in the CDC literature.
- We present and discuss applications of CDC in modern networks such as NFV and edge computing.
- We highlight challenges and discuss promising research directions related to CDC.

For the reader's convenience, we classify the related CDC studies according to the challenges that need to be handled. In particular, the issues are communication costs, straggler effects, and security. As such, readers who are interested in or working on related issues will benefit greatly from our insightful reviews and in-depth discussions of existing approaches, remaining/open problems, and potential solutions. The rest of this paper is organized as follows. Section II introduces the fundamentals of CDC. Section III presents basic CDC schemes. Section IV reviews CDC approaches that have been proposed to reduce communication costs. Section V discusses CDC approaches that have been proposed to mitigate straggler effects. Section VI presents CDC approaches that have been proposed to enhance privacy and security in distributed computing. Section VII discusses applications of CDC. Section VIII highlights important challenges and promising research directions. The structure of the survey is presented in Fig. 1. Section IX concludes the paper. A list of abbreviations commonly used in this paper is given in Table I.

II. FUNDAMENTALS OF CODED DISTRIBUTED COMPUTING

Distributed computing has been an important solution to large-scale, complex computation problems, which involves

TABLE I
LIST OF COMMON ABBREVIATIONS USED IN THIS PAPER

Abbreviation	Description
ARIMA	Auto Regressive Integrated Moving Average
BCC	Batch Coupon's Collector
BGC	Bernoulli Gradient Code
BGW	Ben-Or, Goldwasser, and Wigderson
BPCC	Batch-Processing Based Coded Computing
C3P	Coded Cooperative Computation Protocol
CDC	Coded Distributed Computing
CNN	Convolutional Neural Network
CPGC	Coded Partial Gradient Computation
DAG	Directed Acyclic Graphs
DNN	Deep Neural Network
FRC	Fractional Repetition Coding
HCMM	Heterogeneous Coded Matrix Multiplication
IoT	Internet of Things
LCC	Lagrange Coded Computing
LDPC	Low-Density Parity-Check
LT	Luby Transform
MDS	Maximum Distance Separable
MMC	Multi-Message Communication
MPC	Multi-Party Computation
NFV	Network Function Virtualization
PCR	Polynomially Coded Regression
PDA	Placement Delivery Array
SDMM	Secure Distributed Matrix Multiplication
SGC	Stochastic Gradient Coding
SGD	Stochastic Gradient Descent
SVD	Singular Vector Decomposition
UAV	Unmanned Aerial Vehicle

massive amounts of data. Various distributed computing models, e.g., cluster computing [32], grid computing [33] and cloud computing [29], [34], have been developed to perform the distributed computation tasks while providing high quality of services (QoS) to the users. Among the distributed computing models, cloud computing is gaining much popularity recently as it eliminates the need for users to purchase expensive hardware and software resources since the users only need to pay for the cloud services based on their usage needs in an on-demand basis. A comparison between cluster, grid and cloud computing models is summarized in Table II.

Distributed computing has been widely implemented in a variety of applications, e.g., sensor networks [35], healthcare applications [36], the development of smart cities [37], automated manufacturing processes [38] and vehicular applications [39]. In order to improve the performance of the distributed computing systems, various aspects such as resource allocation strategies [29], task allocation strategies [40], [41], scheduling algorithms [42], [43], incentive mechanisms [44], [45], energy efficiency [32], network security [46] and the performance modeling [47] of the distributed computing systems have been extensively studied in

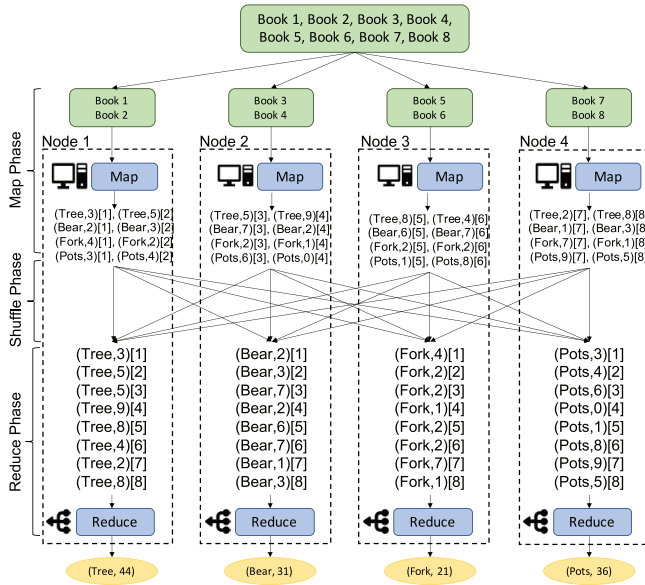


Fig. 2. Illustration of conventional MapReduce framework. The intermediate output pairs are represented by (key, frequency)[book number] and the output pairs are represented by (key, frequency).

the literature. However, the performance of the distributed computing systems is still limited by the high communication costs and straggler effects which lead to a longer time needed to execute the computation tasks. As a result, recent research has focused on coding techniques to overcome these implementation challenges of the distributed computing systems, the aims of which are to minimize the communication load as well as to mitigate the straggler effects.

In this section, we discuss the commonly used distributed computation frameworks for the implementation of coding techniques and algorithms. Note that while the different computation frameworks are useful for different computing applications, we focus specifically on the MapReduce framework [6] as the majority of the research works on CDC schemes are based on the MapReduce computation framework. We also introduce the two main lines of works in CDC, i.e., to reduce communication load and to mitigate the straggler effects, which aim to solve the challenges in distributed computing.

A. Coded Distributed Computation Frameworks

While the distributed computation frameworks have moved beyond a simple MapReduce framework, the majority of the studies on CDC have focused on the MapReduce framework.

MapReduce [6] is a software framework and programming model that runs on a large cluster of commodity machines for the processing of large-scale datasets in a distributed computing environment. The cluster of computers is modeled as a master-worker system which consists of a single master and multiple workers to store and analyze massive amounts of unstructured data. Due to its scalability and its ability to tolerate machines' failure [48], the MapReduce framework is commonly used in a wide range of applications [6], e.g., the analysis of Web access logs, the clustering of documents, the

construction of Web-link graph that matches all source URLs to a target URL and the development of machine learning algorithms. Generally, the MapReduce computation framework involves the processing of a large input file to generate multiple output pairs of which each pair consists of a key and a corresponding value. Figure 2 demonstrates the implementation of the conventional MapReduce framework to determine the frequency of occurrence of 4 specific words in the books, where the 4 workers are to compute the 4 output pairs. There are three important phases in the MapReduce computation framework.

- 1) In the **Map** phase, there are two stages, namely the allocation of Map tasks and the execution of Map tasks. Generally, a Map task is a function that generates a key-value output pair based on the allocated subfiles. Firstly, as seen in Fig. 2, the master splits the input file into 8 subfiles of smaller sizes and allocates the subfiles to the 4 workers. Secondly, each worker produces 4 intermediate key-value pairs for each allocated subfile using the Map functions. Since the workers are allocated 2 subfiles each, each worker generates 8 computed intermediate results.
- 2) In the **Shuffle** phase, the workers exchange their computed intermediate results to obtain the required intermediate results for the computations of the Reduce functions. In particular, in each time slot, one of the workers creates a message that contains information of the intermediate output pairs from the Map phase and transmits the message to other workers. The shuffling process continues until all workers have received the required intermediate output pairs for the Reduce phase.
- 3) In the **Reduce** phase, the workers aggregate the 8 intermediate key-value pairs obtained from the Shuffle phase and compute the final result which is a smaller set of key-value pairs using the Reduce functions. In particular, the reduction tasks are evenly distributed among the workers. Each reduce function is responsible for the evaluation of a key. For example, node 1 in Fig. 2 is responsible for the evaluation of "Tree". Therefore, the total number of Reduce functions needed equals the total number of output keys, i.e., 4 Reduce functions are needed to compute 4 output pairs.

Apart from the MapReduce framework, there are other distributed computation frameworks that provide support for the processing of large-scale datasets such as:

- *Spark* [49]: It supports applications that need to reuse a working dataset across multiple parallel processes. These applications cannot be expressed as efficiently as acyclic data flows which are required in popular computation frameworks such as MapReduce. There are two use cases for the implementation of Spark computation framework: (i) iterative machine learning algorithms which operate on the same dataset repeatedly, and (ii) iterative data analysis tools, where different users query for a subset of data from the same dataset.
- *Dryad* [50]: By allowing the developers to construct their own communication graphs and subroutines at

TABLE II
COMPARISON BETWEEN CLUSTER, GRID AND CLOUD COMPUTING MODELS [29]

Feature	Cluster	Grid	Cloud
Size	Small to medium	Large	Small to large
Network type	Private, LAN	Private, WAN	Public, WAN
Job management and scheduling	Centralized	Decentralized	Both
Coupling	Tight	Loose/tight	Loose
Resource reservation	Pre-reserved	Pre-reserved	On-demand
Service-level agreement (SLA) constraint	Strict	High	High
Resource support	Homogeneous and heterogeneous (GPU)	Heterogeneous	Heterogeneous
Virtualization	Semi-virtualized	Semi-virtualized	Completely virtualized
Security type	Medium	High	Low
Service-oriented architecture and heterogeneity support	Not supported	Supported	Supported
User interface	Single system image	Diverse and dynamic	Single system image
Initial infrastructure cost	Very high	High	Low
Self service and elasticist	No	No	Yes
Administrative domain	Single	Multi	Both

TABLE III
COMPARISON BETWEEN DISTRIBUTED COMPUTATION
FRAMEWORKS [51]

Feature	MapReduce [6]	Spark [49]	Dryad [50]	CIEL [51]
Dynamic Control Flow	No	No	No	Yes
Task Dependencies	Fixed (2-stage)	Fixed (Multi-stage)	Fixed (DAG)	Dynamic
Fault Tolerant	Transparent	Transparent	Transparent	Transparent
Data Locality	Yes	Yes	Yes	Yes
Transparent Scaling	Yes	Yes	Yes	Yes

the vertices through a simple, high-level programming language, Dryad executes large-scale data-intensive computations over clusters consisting multiple computers. It does not require the developers to express their code in Map, Shuffle and Reduce phases in order to adopt the MapReduce framework for computations. Besides, the Dryad execution engine, which is based on the constructed data flow graph, takes care of the implementation issues of the distributed computation tasks such as the scheduling of tasks, allocation of resources and the recovery from communication and computation failures.

- **CIEL [51]:** The main characteristic of the CIEL computation framework is that it allows data-dependent data flows where the directed acyclic graphs (DAG) are built dynamically based on the execution of previous computations, rather than a statistically predetermined DAG. Instead of maximizing throughput, CIEL aims to minimize latency of individual tasks, which is very useful for the implementation of iterative algorithms, where latency grows significantly as the number of iteration increases.

A comparison between the distributed computation frameworks is presented in Table III.

B. Objectives of CDC Schemes

There are two main lines of works in CDC. Firstly, the CDC schemes are implemented to minimize the *communication load* in distributed computing systems. Secondly, the CDC schemes

aim to mitigate *straggler effects* which cause a delay in the computation of the distributed tasks. For each of the objectives, we discuss the importance of solving these issues to improve the performance of the distributed computing systems. Then, we briefly discuss the prior solutions that have been proposed in the literature to meet these objectives. However, the prior solutions do not adopt coding approaches. Different from the prior solutions, the CDC schemes are able to meet these objectives by introducing coded redundancy. In fact, the CDC schemes outperform the replication methods, e.g., naive replication [18] and fork-join model [14], [52], which introduce redundancy without coding techniques, in terms of time taken to execute the tasks.

1) **Communication Load:** Among the three phases of the MapReduce computation framework, the Shuffle phase dominates the time required to complete the computation tasks [7], [8], [53] since multiple communications between the computing nodes are needed to exchange their intermediate results. For the Hadoop cluster at Facebook, it is observed that on average, the data shuffling phase accounts for 33% of the overall job execution time [9]. In fact, the data shuffling phase is more time consuming when running on heterogeneous clusters with diverse computational, communication and storage capabilities. When running TeraSort [54], which is a conventional distributed sorting algorithm for large amounts of data, and Self-Join applications on heterogeneous Amazon EC2 clusters, 65% and 70% of the overall job execution time is spent on the Shuffle phase respectively [10]. The data shuffling process is also an important step in implementing distributed learning algorithms. In particular, to train machine learning models with distributed algorithms, it is common to shuffle the data randomly and run the algorithms iteratively for a number of times such that the computing nodes compute a different subset of the data at each iteration until there is a convergence [55]–[57]. For the logistic regression application which requires at least 100 iteration to converge, 42% of the iteration time is spent on communication [9]. For each time the data shuffling process is performed, the entire training dataset is communicated over the network, resulting in high communication costs which limit the performance of the distributed computing systems.

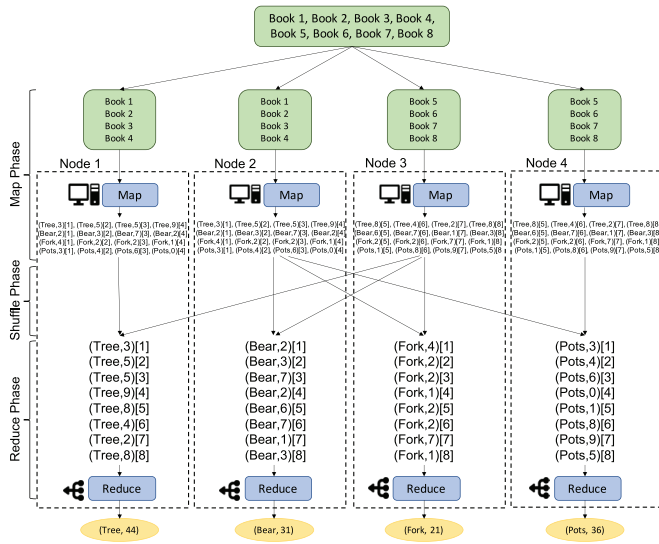


Fig. 3. Illustration of naive replication MapReduce framework.

Since the performance of the data shuffling process has a significant impact on the overall performance of the distributed computing systems, it has been extensively studied in the literature, e.g., [58]–[65]. Various data shuffling strategies are proposed to achieve different objectives such as minimizing the job execution time, maximizing the utilization of resources and accommodating interactive workloads. While the overlap between the Map computations and the Shuffle communication helps to reduce the latency of the distributed computation tasks [58], the computing nodes require large storage capacities for buffering. An efficient and adaptive data shuffling strategy is proposed in the study of [59] to manage the trade-off between the accumulation of the shuffle blocks and the minimization of the utilization of memory space to reduce the overall job execution time and improve the scalability of the distributed computing systems. In [60], the authors propose a virtual data shuffling strategy which reduces storage space and traffic load in the network by delaying the actual movement of the data until it is needed to complete the computations in the Reduce phase.

To improve the performance of the data shuffling process, task scheduling algorithms such as Quincy scheduler [61], Hadoop Fair Scheduler [64] and delay scheduling algorithm [65] are also designed to allocate tasks to the workers. In the design of optimal task scheduling and task selection algorithms, the communication load can be minimized through various approaches such as by optimizing the placement of computation tasks, distributing the computing resources fairly to the nodes and maximizing the resource utilization of the systems. Since the task scheduling schemes are not the focus of this survey, we refer interested readers to the study of [66] and the references therein for more detailed information on the scheduling techniques.

One of the ways to reduce communication load in the shuffling phase is by repeating the computation tasks [18]. Figure 3 illustrates the implementation of the MapReduce framework by using the naive replication method in which 4 computing

nodes are required to compute 4 output pairs, which is the same computation task illustrated in Fig. 2. By simply replicating the Map tasks where each worker is required to compute more intermediate output pairs, i.e., 16 in Fig. 3 instead of 8 intermediate output pairs in Fig. 2, the communication load is reduced as fewer intermediate output pairs are communicated. For example, in the conventional MapReduce framework in Fig. 2, node 1 needs to obtain 6 intermediate output pairs from other workers whereas in the naive replication scheme in Fig. 3, node 1 only needs to obtain 4 intermediate output pairs from other workers.

However, the aforementioned non-coding methods have limits to which the communication load in the data shuffling phase can be minimized. Given that the naive replication method reduces communication load in the data shuffling phase by introducing redundancy to the systems (which is also discussed in Section III-A), coding techniques can be used to introduce redundancy to further minimize communication load, which will be discussed in-depth in Section IV.

2) *Straggler Effects*: In distributed computing systems, the computing nodes may have heterogeneous computational capabilities, i.e., different processing speeds. As such, another line of work in the CDC literature is to solve the bottleneck that results from a variation in time taken to complete the allocated tasks. In distributed computing systems, there are *stragglers*, which are the computing nodes that run unexpectedly slower than the average or nodes that may be disconnected from the network due to several factors such as insufficient power, contention of shared resources, imbalance work allocation and network congestion [67], [68]. As a result, the overall time needed to execute the tasks is determined by the slowest computing node. We briefly discuss the prior approaches (which are summarized in Table IV) to handle the straggler effects as follows.

- *Stragglers Detection*: The most direct approach to mitigate the straggler effects is to detect the stragglers and act on them early in their lifetime. For example, Mantri [68] detects the stragglers by identifying the tasks that are processed at a rate slower than the average. The system determines the cause of the delay and implements targeted solutions to mitigate the stragglers. The solutions include restarting the tasks allocated to the stragglers at other computing nodes, optimally allocating the tasks based on network resources as well as protecting against interim data loss by replicating the outputs of valuable tasks.
- *Work Stealing* [69]: The basic idea of the work stealing algorithm is to allow the faster computing nodes to take over the remaining computation tasks from the slower computing nodes so that the overall job execution time is minimized. By adopting this approach, the faster computing nodes operate continuously while leaving the slower computing nodes idle after their jobs are taken over by the faster computing nodes till the end of the computation session.
- *Work Exchange* [13]: By leveraging on the information of computational heterogeneity in the system, the master first allocates the tasks to the workers based on their

TABLE IV
APPROACHES TO MITIGATE THE STRAGGLER EFFECTS

Approach	Key Ideas	Shortcomings
Stragglers Detection	Detects the stragglers, determines the cause of delay and implements targeted solutions	Difficult to identify the cause of delays
Work Stealing	Reallocates the remaining computation tasks from the slower workers to the faster workers	The capabilities of the slower workers are not maximized
Work Exchange	Reallocates the computation tasks every time a worker completes its tasks	Incurs high communication costs due to the feedback from the workers to the master as well as the reallocation of data
Naive Replication	Introduces redundancy where each computation subtask is performed by more than one computing node	Incurs high communication costs and computation load
Coded Redundancy	Uses coding techniques to introduce redundancy such that the master can recover the final result from any decodable set of workers	Still incurs high communication costs and computation load, but lower than that of naive replication

computational capabilities. Upon receiving the first computed result from any of the workers, the master pauses the computation process and redistribute the remaining incomplete work to be computed by the workers. The process is performed for a number of iterations until all work is done. Since the workers need to inform the master of the amount of work done at each time that the computation process is paused, additional communication costs are incurred. The higher communication costs are also a result of the reallocation of data to the workers.

- *Naive Replication*: One of the solutions to handle stragglers in the distributed computing systems is by introducing redundancy to minimize computation latency. The computation task is replicated and executed over multiple computing nodes. Since all computing nodes are working on the same computation task, the time required to complete the computation task is determined by the fastest computing node. The partial computations of the remaining computing nodes are discarded. Experiments on Google Trace data [14] have shown the effectiveness of the use of redundancy in minimizing computation latency by eliminating the need for the computed results by the stragglers. However, the introduction of redundancy comes at the expense of higher cost such as high communication bandwidth and high computation load [14], [52], [70]–[73]. Various redundancy strategies have been analyzed to derive the limiting distribution of the state of the systems [52], [70]. Although the introduction of redundancy helps to reduce latency, the performance varies under different settings. In fact, in some settings, it is optimal to not use any redundancy strategy. Looking into this, the work in [71] presents the optimal redundant-requesting policies under diverse settings.

Similar to the prior works to reduce communication costs as discussed in the previous subsection (Section II-B1), the prior works to mitigate the straggler effects do not adopt coding approaches. Coding techniques can also be used to introduce redundancy into the systems to mitigate the straggler effects. The authors in [72], [73] investigate the tradeoff between latency and cost for both replication-redundancy systems and coded-redundancy systems. Coded-redundancy systems outperform the replication-redundancy systems in

both latency and cost. In other words, by using coding techniques, the latency and cost incurred are lower than that of naive replication. The use of coding techniques to mitigate the straggler effects is discussed in more detail in Section III-B and Section V. To better understand the CDC schemes, some of the commonly used performance metrics of the distributed computing systems are defined as follows.

- 1) *Storage Space* is defined as the total number of files stored across K computing nodes, normalized by the total number of subfiles N [74].
- 2) *Computation load* is represented by r , where $1 \leq r \leq K$, is defined as the total number of Map functions computed across K computing nodes, normalized by the total number of subfiles N [17]. In particular, when $r = 1$, each Map function is only computed by a single computing node. When $r = 2$, each Map function is computed by two computing nodes on average.
- 3) *Communication load* is represented by L , where $0 \leq L \leq 1$, is defined as the total number of bits communicated by the K computing nodes in the Shuffle phase, normalized by the total number of subfiles N [17].

Given that coding techniques are able to solve the aforementioned implementation challenges of the distributed computing systems, we review various proposed CDC schemes, which is the main focus of this paper. In the following section, we present a tutorial of the simple CDC schemes along these two lines of works, i.e., minimizing communication load and mitigating the straggler effects, of which is useful to better understand the related works discussed in Sections IV, V, and VI.

III. CODED DISTRIBUTED COMPUTING (CDC) SCHEMES

Recently, coding techniques have become a popular approach to solve the challenges of the distributed computing systems. As mentioned previously, there are two main lines of work in CDC: (i) to reduce the communication costs and (ii) to mitigate the straggler effects. In this section, we introduce the two basic CDC schemes which are the first works that show the effectiveness of using coding techniques to solve these two challenges separately. Then, we discuss a unified CDC scheme that characterizes the tradeoff between computation latency and communication load.

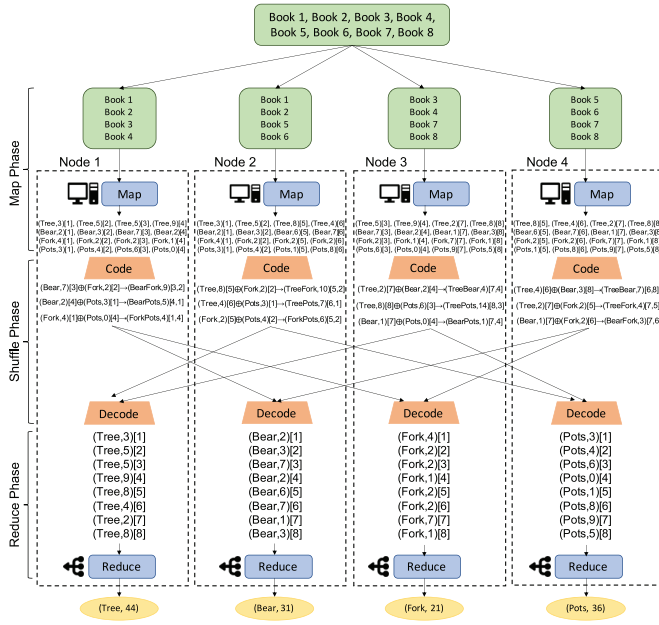


Fig. 4. Illustration of Coded MapReduce framework.

A. Basic CDC for Multicasting Opportunities

In the conventional MapReduce computation framework as shown in Fig. 2, after the split of the input file into multiple subfiles, each subfile is mapped to only one of the workers. The naive replication scheme, i.e., uncoded data shuffling scheme which relaxes this restriction, can reduce the communication costs of the system by allowing each subfile to be replicated and mapped to more than one worker. In the example illustrated in Fig. 3, each subfile is repeated twice. Hence, as compared to the conventional MapReduce framework, each worker has more Map tasks to perform in the naive replication scheme. However, by simple replication, the communication load in the Shuffle phase decreases and this gain is known as the *repetition gain*. Specifically, the communication load for uncoded schemes which include both the conventional MapReduce framework and the naive replication scheme, is denoted as follows [17]:

$$L_{\text{uncoded}}(r) = 1 - \frac{r}{K}, \quad (1)$$

where r is the computation load of each worker, which can be interpreted as the average number of workers that compute each Map function, and K is the number of workers in the network. Based on Equation (1), the communication loads achieved by the conventional MapReduce framework in Fig. 2 and the naive replication scheme in Fig. 3 can be computed. Specifically, the number of workers K is 4 for both cases. For the conventional MapReduce framework in Fig. 2, the computation load $r = 1$ since each Map function is computed only once, whereas for the naive replication scheme in Fig. 3, the computation load $r = 2$ since each Map function is computed twice. As such, the communication loads are $\frac{3}{4}$ and $\frac{1}{2}$ for the conventional MapReduce framework and the naive replication scheme respectively.

To further reduce the communication load, i.e., to increase the repetition gain, the Coded MapReduce computation framework is proposed in [18] where the Map tasks are carefully distributed among the workers and the messages are encoded for transmission in the Shuffle phase by using coding theory. Figure 4 illustrates the Coded MapReduce framework with 4 workers to determine the 4 output pairs. After the Map phase, the worker multicasts a bit-wise XOR of two computed intermediate pairs, denoted by $\oplus$, satisfying the requirements of two other workers simultaneously. For example, node 1 multicasts a bit-wise XOR of “Bear” and “Fork” to both nodes 2 and 3, which involves the transmission of only one packet of information, instead of two packets if the information is sent separately to the nodes in a unicast manner. Since the intermediate output pairs are now coded, there is an additional step of decoding before the Reduce functions are applied. Given the coded “BearFork” information, node 2 is able to decode and recover the required “Bear” information by canceling the “Fork” information since node 2 has also computed the same “Fork” information. Similarly, node 3 can recover “Fork” information by canceling the “Bear” information. The Coded MapReduce [18] scheme has shown its effectiveness by reducing the communication load by 66% and 50% as compared to the conventional MapReduce framework and the naive replication scheme respectively. Note that the concept of using coding techniques to reduce communication load in the distributed computing systems is similar to that of network coding [75]. However, network coding aims to ensure reliable data transfer, storage, and access in the distributed storage systems whereas CDC aims to optimize the computation and communication phases of the distributed computing systems.

Since the reduction of communication load comes at the expense of the increase of computation load, a more generalized framework known as the Coded Distributed Computing (CDC) scheme is introduced in [17]. The study of [17] presents the fundamental inverse relationship between computation load and communication load. Specifically, the communication load in the Shuffle phase can be reduced by a factor r by increasing the computation load in the Map phase by the same factor r , as shown in Fig. 5. The communication load achieved by the CDC framework, L_{coded} , is given as follows:

$$L_{\text{coded}}(r) = \frac{1}{r} \left(1 - \frac{r}{K} \right). \quad (2)$$

Note that the information-theoretic lower bound derived on the minimum communication load $L^*(r)$ equals $L_{\text{coded}}(r)$ of the CDC framework. As such, the optimal tradeoff between the computation load and communication load [17] is characterized by $L^*(r) = L_{\text{coded}}(r)$, $r \in \{1, \dots, K\}$.

From Equation (1) which applies to the uncoded computation schemes, the communication load L decreases linearly as the computation load r increases. However, when the number of workers K becomes large, there is no significant impact of increasing computation load on the communication load. On the other hand, for the CDC framework, the communication load is inversely proportional to the computation load (Equation (2)). Even when K becomes large, the increase in

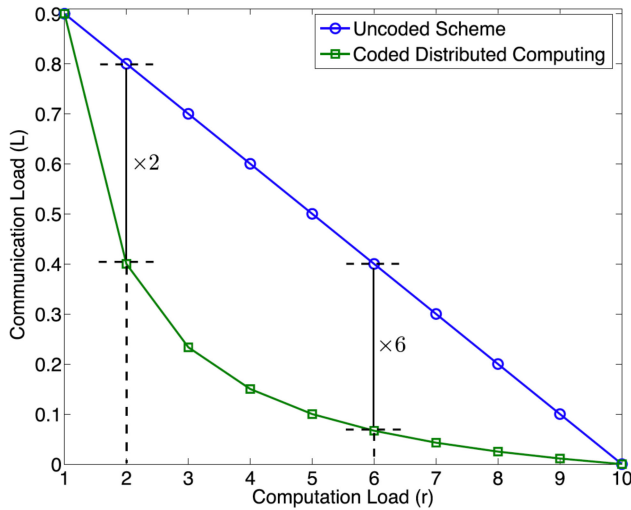


Fig. 5. Comparison of communication load between the CDC scheme and the uncoded scheme, given the number of computing nodes $K = 10$ [17].

computation load still significantly reduces the communication load. Building on the work of [17], the authors in [76] have proposed a flexible low complexity design for the CDC networks in order to support the different intermediate results of the workers. This offers flexibility in adapting to the varying network parameters, e.g., number of workers and amount of computation load.

Since the CDC framework can be applied to any distributed computation framework with an underlying MapReduce structure, the performance of the CDC framework on TeraSort [54] is evaluated. Experimental results on Amazon EC2 clusters show that the Coded TeraSort scheme [77] which is a coded distributed sorting algorithm, achieves a reduction in the overall job execution time by factors of 2.16 and 3.39 with 16 workers and computation loads of $r = 3$ and $r = 5$, respectively as compared to the uncoded TeraSort scheme.

Previously in [17], computation load is linearly dependent on the number of replicated Map tasks, i.e., load redundancy, as each worker is assumed to compute all intermediate values for all subfiles allocated in its memory. However, the workers can be selective in choosing the intermediate values to compute. As such, the load redundancy is no longer a direct measure of computation load. In other words, the storage constraints do not necessarily imply computation constraints. Building on the work in [17], the authors in [78] propose an alternative tradeoff between computation load and communication load under a predefined storage constraint. In fact, the computation load is quadratic in terms of load redundancy. By taking load redundancy into consideration, an alternative computation-communication tradeoff curve is derived. In particular, this alternative tradeoff curve is especially relevant to the workers that do have the sufficient resources or time to perform computations for all the allocated subfiles. Given that the workers can only perform a limited amount of computations below the computation load threshold, the alternative tradeoff curve proposed in [78] accurately defines the amount of communication load needed for the distributed computation tasks.

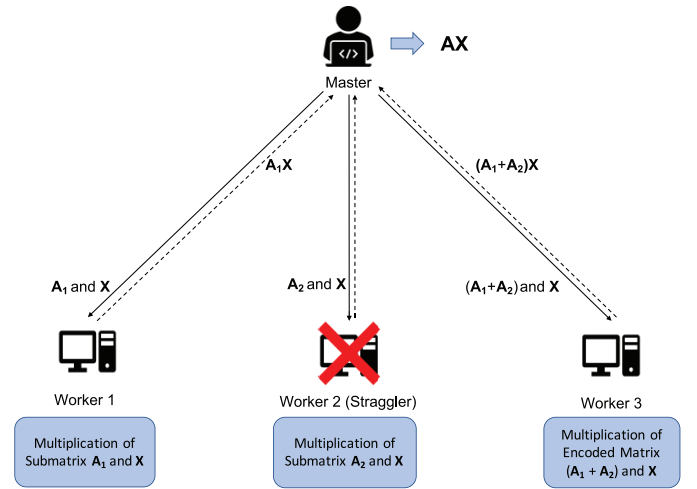


Fig. 6. Illustration of coded computation with 3 workers. The master is able to recover the final result upon receiving the computed results from any 2 workers.

Since the workers are not required to compute all intermediate results that can be obtained from their locally stored data, the storage capabilities of the workers should be considered in the CDC computation framework in [17]. The study of [74] characterizes the tradeoff between storage, computation and communication where the minimum communication load is determined given the storage and computational capabilities of the workers. In particular, the optimal computation curve is obtained by characterizing the optimal storage-communication tradeoff given the minimum computation load. As a result, the triangles between the optimal communication curve and the optimal computation curve reflect the pareto-optimal surface of all achievable storage-computation-communication triples. However, as the number of workers in the system increases, the number of input files required increases exponentially, resulting in an increase in the number of transmissions needed and hence high communication costs. As such, it is important to reduce the number of input files, which is discussed in Section IV-A3 later.

B. Basic CDC for Recovery Threshold Minimization

Apart from reducing communication load in the Shuffle phase of the MapReduce framework, coding techniques can also be used to alleviate the straggler effects. Since matrix multiplication is one of the most basic linear operations used in distributed computing systems, a coded computation framework is proposed in [19] to minimize computation latency of distributed matrix multiplication tasks. The coded computation framework uses erasure codes to generate redundant intermediate computations. In particular, the master encodes the equal-sized data blocks, i.e., submatrices and distributes them to the workers to compute the local functions. Upon completion, the workers transmit the computed results to the master. The master can recover the final result by using the decoding functions once the local computations from any of the decodable sets are completed.

TABLE V
CODING TECHNIQUES TO MITIGATE THE STRAGGLER EFFECTS

Problems	Ref.	Coding Schemes	Key Ideas
Matrix-Vector	[19]	MDS Codes	Reduce the computation latency as the master is able to recover the final result without waiting for the slowest computing node
	[79]	LT Codes	Exploit the rateless property to generate unlimited number of encoded symbol from a finite set of source symbols
	[80]	Short-Dot Codes	Reduce the length of dot-products computed at the computing nodes by introducing sparsity to the encoded matrices
	[81]	s-diagonal codes	Exploit the diagonal structure of the matrices to achieve both optimal recovery threshold and optimal computation load
Matrix-Matrix	[82]	Product Codes	Instead of encoding the matrices along one dimension as in the MDS-coded schemes, the matrices are encoded with MDS codes along both dimensions, i.e. row and column
	[83]	Polynomial Codes	- Design the algebraic structure of the encoded matrices such that the MDS structure is found in both the encoded matrices and the intermediate computations - Reconstruct the final results by solving the polynomial interpolation problem
	[84]	MatDot Codes	Achieve lower recovery threshold than Polynomial Codes [83] at the expense of higher communication costs by computing only the relevant cross-products
	[84]	PolyDot Codes	Characterize the tradeoff between recovery threshold and communication costs where Polynomial Codes [83] and MatDot Codes are the two extreme ends on this tradeoff curve
	[85]	Sparse Codes	Exploit to sparsity in both input and output matrices to reduce computation load, while achieving near optimal threshold
Gradient Descent	[86]	Fractional Repetition Coding	- Workers are divided into multiple groups and data is divided among the workers in each group - Each partition of data is performed by more than one worker
	[86]	Cyclic Repetition Coding	Data is allocated based on a cyclic assignment strategy
	[87]	Cyclic MDS Codes	The entries to the columns of the encoding matrix are cyclic shifts of the entries to the first column
	[88]	Reed-Solomon Codes	Use balanced mask matrix and chooses appropriate codewords from the RS codes to construct the encoding matrix
	[89]	Batch Coupons Collector	- Divide the data into multiple batches which are allocated randomly to the workers for computations - communication costs are significantly reduced as there is no need for communication between the workers and for feedback from the master to the workers
	[90]	Polynomially Coded Regression	Encode the data batches directly instead of the computed intermediate results
Convolution	[91]	Coded Convolution	Split both vectors into multiple parts of specified length and encodes one of the vectors with MDS codes
Fourier Transform	[92]	Coded Fourier Transform	Leverage on recursive structure and the linearity of the discrete Fourier Transform operations

As an illustration, in Fig. 6, the master aims to compute the final result of $\mathbf{AX}$. In order to do so, the master needs to collect the computed results of both $\mathbf{A}_1\mathbf{X}$ and $\mathbf{A}_2\mathbf{X}$. Even if worker 2 is a straggler, the master is able to recover the final result upon receiving the computed results from workers 1 and 3. Specifically, the master obtains the computed result of $\mathbf{A}_2\mathbf{X}$ by subtracting $\mathbf{A}_1\mathbf{X}$ from $(\mathbf{A}_1 + \mathbf{A}_2)\mathbf{X}$ that are received from workers 1 and 3 respectively. As a result, the master does not rely on the stragglers to recover the final result. This is different from traditional distributed computing systems without coding techniques where the master needs to collect the computed results from all workers, including the stragglers. As such, with CDC, the total computation is not determined by the slowest straggler, but by the time when the master receives computed results from some decodable set of indices. In this work [19], the authors explore the effectiveness of encoding the submatrices by using maximum distance separable (MDS) codes [93] to mitigate the effects of stragglers.

In [19], considering K workers and a shifted-exponential distribution for the job execution time of the distributed algorithm, the simulation results show that the optimal repetition-coded distributed algorithm achieves a lower average job execution time when the straggling parameter is smaller than one, i.e., $\mu < 1$ but is still slower than the optimal MDS-coded distributed algorithm by a factor of $\Theta(\log K)$. However, the storage cost of the coded distributed algorithm is higher than that of the uncoded distributed algorithm as more data is required to be stored at the workers' sites for the coded distributed algorithm. In addition, the proposed algorithm is tested on an Amazon EC2 cluster and is compared against various parallel matrix multiplication algorithms, e.g., block matrix multiplication, column-partition matrix multiplication and row-partition matrix multiplication. From the simulation results, the authors show that the proposed algorithm in [19] performs better where the coded matrix multiplication achieves 40% and 39.5% reduction in average job execution time on clusters of m1-small and c1-medium instances with 10 workers

each respectively as compared to the best of the three uncoded distributed algorithms.

Although the MDS codes proposed in [19] is able to mitigate the straggler effects, it cannot be generalized to all types of computation tasks. In order to mitigate the straggler effects of different distributed computation tasks, the coding techniques can be designed by exploiting the algebraic structures of the specific operations. An important performance metric that is introduced in the CDC schemes is the recovery threshold, which refers to the worst-case required number of workers the master needs to wait to recover the final result for job completion [83]. The smaller the recovery threshold, the shorter the computation latency. The objective is to reduce the recovery threshold so that the final result can be recovered by waiting for a smaller number of workers, thus contributing to a reduction in computation latency. Here, we discuss the coding techniques for various types of computation tasks, namely (i) matrix-vector multiplications, (ii) matrix-matrix multiplications, (iii) gradient descent, (iv) convolution and Fourier transform. Table V summarizes the coding techniques designed for different distributed computation tasks.

1) *Matrix-Vector Multiplications*: Distributed matrix-vector multiplications are the building blocks of linear transformation computations which are an important step in machine learning and signal processing applications. In particular, the computations of linear transformation on high-dimensional vectors are required for popular dimensionality reduction techniques such as Linear Discriminant Analysis (LDA) [94] and Principal Component Analysis (PCA) [95].

Instead of using MDS codes that is proposed in [19], the authors in [79] propose the use of Luby Transform (LT) codes to mitigate the straggler effects in distributed matrix-vector multiplications problems. Different from the works in [96] and [85] which use LT codes in fixed-rate settings, the rateless property of the LT codes can be exploited to generate unlimited number of encoded symbol from a finite set of source symbols. There are several advantages of using rateless codes: (i) near-ideal load balancing, (ii) negligible redundant computation, (iii) maximum straggler tolerance, and (iv) low decoding complexity. To further reduce the latency for practical implementations, blockwise communication can be used to transmit the submatrix-vector products. Instead of transmitting each encoded row-vector product separately to the master, the workers are allowed to transmit the computed results in blocks where each block comprises a few row-vector products, reducing the number of communication rounds needed and hence minimizing the time needed to complete the computation tasks.

The authors in [80] propose Short-Dot codes to perform the computations of linear transforms reliably and efficiently under the presence of stragglers. Specifically, the workers compute shorter dot products by imposing sparsity on the encoded submatrices. However, there is tradeoff between the optimal threshold recovery and the length of the dot-products where the master needs to wait for computed results from more workers when the length of the dot-products is shorter. The experimental results on the classification of hand-written digits of MNIST show that the Short-Dot codes achieve 32% faster expected computation time than the MDS codes [19].

Although the Short-Dot codes [80] can offer lower recovery threshold, the greater length of the dot-products means greater computation load for the computing nodes. With this concern, s-diagonal codes [81] are proposed to achieve both optimal recovery threshold and optimal computation load by exploiting the diagonal structure of the encoding matrix. The computation time can be further reduced by using a low-complexity hybrid decoding algorithm which combines the peeling decoding algorithm and Gaussian elimination techniques.

2) *Matrix-Matrix Multiplications*: For large-scale distributed matrix-matrix multiplications, the coded computation schemes based on MDS codes are no longer suitable as the encoding and decoding processes scale with system size. Besides, the size of one of the matrices is assumed to be small enough in order to allow individual workers to perform the computations [82], restricting the implementation of MDS codes in large-scale multiplications. Hence, for large-scale problems, coded schemes not only need to achieve low computation time, but also require efficient encoding and decoding algorithms in order to minimize the overall job execution time. To deal with the straggler effects in high-dimensional distributed matrix multiplications, four types of coded computation schemes are introduced.

- *Product codes [82]*: Product codes are implemented by building a larger code upon smaller MDS codes. Instead of encoding computations along only one dimension in MDS-coded schemes, the product codes encode computations along both dimensions, i.e., rows and columns of the matrices. When the number of backup workers increases sub-linearly with the number of subtasks, the product-coded schemes outperform the MDS-coded schemes in terms of average computation time and decoding time. In the linear regime, the one-dimensional decoding of the MDS-coded schemes is sufficient to recover the missing entries of the computation results. By allowing each row and column of the MDS constituent codes to have different code rates [97], the average computation time can be further reduced, contributing to a decrease in the overall job execution time. Product codes can also be used to solve higher-dimensional linear operations such as tensor operations by exploiting the tensor-structured encoding matrix [98]. To reduce the decoding time of the product codes, efficient decoding algorithms such as Reed-Solomon codes and LDPC codes can be explored.
- *Polynomial codes [83]*: The key advantage of the polynomial codes in solving large-scale matrix multiplication problems is that they provide a lower bound to the optimal recovery threshold. For polynomial codes, the recovery threshold does not scale with the number of workers involved whereas for MDS codes and the product codes, the recovery thresholds scale proportionally with the number of workers. By taking advantage of the algebraic structure of the polynomial codes, the master can recover the final result by using polynomial interpolation algorithms, e.g., the Reed-Solomon codes, to decode the computation results from the workers. In addition to the optimal recovery threshold, the polynomial-coded schemes achieve minimum possible

computation latency and communication load for distributed matrix multiplications. However, as the number of workers increases, the encoding and decoding costs are much higher than that of the product codes. Furthermore, by implementing Reed-Solomon codes, there is a limit to the number of workers that can be handled, which is not useful for practical implementations where the systems may involve up to thousands of nodes. As an extension to the polynomial codes proposed in [83], the entangled polynomial code that is proposed in [99] achieves a lower recovery threshold which is only half of that achieved by the PolyDot codes [84], to be discussed later. Different from the polynomial codes which only allow column-wise partitioning of the matrices, the entangled polynomial codes allow arbitrary partitioning of the input matrices and evaluate only a subspace of bilinear functions such that unnecessary multiplications are avoided. The issue of numerical stability has also received attention to ensure the scalability of the polynomial-coded schemes [100].

- *PolyDot codes [84]*: PolyDot codes characterize the tradeoff between the recovery threshold and communication costs where the polynomial codes and the MatDot codes are special instances of this coding framework by considering two extreme ends of this tradeoff: minimizing either recovery threshold or communication costs. In particular, the MatDot codes achieve lower recovery threshold than the polynomial codes at the expense of much higher communication costs. This is achieved by only computing the relevant cross-products of the submatrices. Building on the work of PolyDot codes [84], the Generalized PolyDot codes [101] are used to compute matrix-vector multiplications and achieve the same recovery threshold as the entangled polynomial codes [99]. More importantly, the Generalized PolyDot codes can be extended for the training of large deep neural networks (DNNs), which consists of multiple non-linear layers.
- *Sparse codes [85]*: Although the polynomial codes [83] achieve optimal recovery threshold, the computation loads of the workers increase due to the increased density of the input matrix, resulting in an increase in the overall job execution time which is not desirable. By exploiting sparsity, i.e., the number of zero entries of the encoded matrix, not only the recovery threshold is kept low, but the computation loads of the workers also decrease while maintaining a nearly linear decoding time [102], jointly contributing to the shorter overall job execution time. The basic idea of the algorithm proposed in [102] is to allow the master to find the linear combination of row vectors such that only the particular relevant sub-blocks are recovered. Then, the entire block of matrix can be recovered by aggregating the partial recovery of sub-blocks. Through simulations, the authors show that the sparse codes require an overall shortest time to complete the job as compared to other computation schemes, e.g., uncoded scheme, product codes [82], polynomial codes [83], sparse MDS codes [80] and LT codes [79].

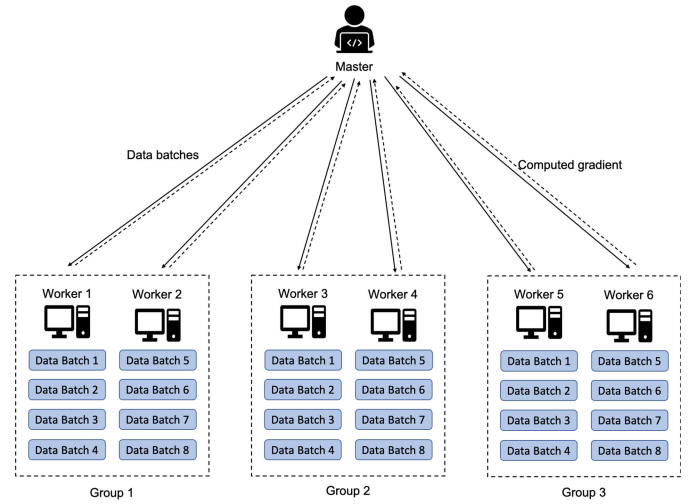


Fig. 7. Fractional repetition coding with 6 workers and 2 stragglers.

A further analysis of the different components of sub-tasks, i.e., communication time, computation time and decoding time, shows that the sparse codes require much shorter time to decode, thus contributing significantly to the shorter overall job execution time.

3) *Gradient Descent*: Apart from matrices, coding techniques can be applied to recover batch gradients of any loss function of the distributed gradient descent tasks. In [86], the authors have introduced the idea of gradient coding which is useful to mitigate stragglers that may slow down the computation tasks. Two gradient coding schemes are proposed, namely (i) fractional repetition coding (FRC) and (ii) cyclic repetition coding. In the FRC scheme, the workers are first divided into several groups. In each group, the data is equally divided and allocated to the workers. As a result, all groups of workers are replica of each other as shown in Fig. 7. Upon completing their subtasks, the workers in each group transmit the sum of partial gradients to the master. In the cyclic repetition coding scheme, the data partitions are allocated to the workers based on a cyclic assignment strategy. The partial gradients computed by each worker are encoded by linearly combining them, of which is transmitted as a single coded message to the master. By applying the gradient coding schemes, the distributed computation tasks do not suffer from delays incurred by the stragglers as the master is able to recover the final result with the results from the non-stragglers. Other coding theories such as the cyclic MDS codes [87] and the Reed-Solomon codes [88] can be used to compute exact gradients of the distributed gradient descent problems.

To efficiently mitigate the straggler effects in distributed gradient descent algorithms, Batch Coupon's Collector (BCC) scheme is proposed in [89]. In BCC, there are two important steps, namely (i) batching and (ii) coupon collecting. In batching, the training set is partitioned into batches which are distributed to the workers randomly whereas in coupon collecting, the master collects the computed results from the workers until the results from all batches of data are received. This decentralized BCC scheme does not require any communication between the workers nodes and each worker is

allocated data batches independently of other workers. As a result, it is easy to implement the BCC scheme in practical scenarios. Another important advantage of the BCC scheme is its universality. Different from other coding schemes that are designed to guarantee their robustness to a fixed number of stragglers, the BCC scheme does not require any prior knowledge about the stragglers which is more practical as it is difficult to estimate the number of stragglers present in the clusters. Furthermore, the BCC scheme can be easily extended to solve gradient descent problems over heterogeneous clusters where the workers have different computational and communication capabilities. The authors show that the BCC scheme speeds up the overall job execution time by up to 85.4% and 69.9% over the uncoded scheme and the cyclic repetition coding scheme [86] respectively.

The gradient coding schemes proposed in [86] illustrate the tradeoff between computation load and straggler tolerance. However, in non-linear learning tasks, communication costs dominate the overall job execution time as the number of iterations increases. As such, to generalize the coding schemes in [86], the authors in [103] incorporate communication costs into their framework and present a fundamental tradeoff between the three parameters, namely computation load, straggler tolerance and communication costs. In particular, for a fixed computation load, the communication costs can be reduced by waiting for more workers.

Instead of encoding the partial gradients computed based on uncoded data as seen in the studies of [86]–[88], coding techniques can be applied directly to the data batches to reduce the straggler effects and the overall job execution time. Considering the gradient computations for least-square regression problems, the polynomially coded regression (PCR) scheme [90] exploits the underlying algebraic property to generate coded submatrices such that they are linear combinations of the uncoded input matrices. The master can evaluate the final gradient by interpolating the polynomials from the computed partial gradients by the workers. Compared to the gradient coding schemes proposed in [86], the simulation results show that the PCR scheme achieves much lower recovery threshold and hence shorter computation and communication time, resulting in shorter time needed for overall job execution.

4) *Convolution and Fourier Transform*: The polynomial codes proposed in both the studies of [83] and [99] can be extended to the applications of distributed coded convolution based on the coded convolution scheme proposed in [91]. The work in [91] explores the use of MDS codes to encode the pre-specified vectors such that fast convolution is performed under deadline constraint. In addition to that, MDS codes can be used to mitigate the straggler effects in widely-implemented distributed discrete Fourier Transform operations [92], which are used in many applications such as machine learning algorithms and signal processing frameworks.

C. Unified CDC Scheme

Given the aforementioned coding schemes of [17] and [19], we can observe that coding techniques are used to optimize

the distributed computing applications with two different approaches. On one hand, the authors in [17] propose the “Minimum Bandwidth Code” that minimizes the communication load by repeating the computation tasks in the Map phase to introduce multicasting opportunities in the Shuffle phase. On the other hand, the authors in [19] propose the “Minimum Latency Code” which minimizes the computation latency by encoding the Map tasks such that the master is able to recover the final result without waiting for the straggling computing nodes.

Inspired by the aforementioned approaches, a unified coded scheme that characterizes the tradeoff between computation latency and communication load is proposed in [104] given the computation load. The coding schemes in [17] and [19] are considered to be the extreme cases of this unified scheme. The unified coded scheme exploits the advantages of the two coding approaches by applying the MDS codes to the Map tasks and replicating the encoded Map tasks. Specifically, the unified coded scheme first encodes the rows of the matrix, following which the coded rows of the matrix are replicated and stored at the computing nodes in a specific pattern. Then, the computing nodes perform the computation until a certain number of the fastest computing nodes complete their tasks. To reduce communication load in the Shuffle phase, coded multitasking is used to exchange the intermediate results that are needed to recover the final results in the Reduce phase. An improvement to the latency-communication tradeoff presented in the unified coded scheme [104] is proposed in [105] by leveraging on the redundancy created by the repetition code. By increasing the redundancy rate of the repetition code, both the communication load in the Shuffle phase and the computation latency in the Map phase can be simultaneously improved, thus contributing to a improved latency-communication tradeoff.

The aforementioned initial works of coding schemes have shown their effectiveness in minimizing communication costs and alleviating the straggler effects. In the following sections, we review related works that leverage on coding techniques to address the implementation challenges of the distributed computing systems.

IV. MINIMIZATION OF COMMUNICATION LOAD

With more computing nodes that are equipped with greater capabilities to collect and process data, massive amounts of data are generated for the computations of user-defined computation tasks. Since the computations are scaled out across a large number of distributed computing nodes, large number of intermediate results need to be exchanged between the computing nodes in the Shuffle phase of the MapReduce framework to complete the computation tasks, resulting in significant data movement. Oftentimes, for the training of a model with distributed learning algorithms, data is shuffled at each iteration, contributing to high communication costs, which is a bottleneck of the distributed computing systems. As a result, there is a need to reduce the communication costs in order to speed up the distributed computation tasks. In this section, we present four approaches to reduce communication costs.

- *File Allocation:* In this approach, the studies aim to design an optimal file allocation strategy that considers the heterogeneities of the computing nodes' capabilities in the systems, maximizing data locality or reducing subpacketization level, which refers to the number of subfiles generated [106], [107]. These different approaches work towards reducing the communication load in the distributed computing systems.
- *Coded Shuffling Design:* Since data shuffling phase incurs a large proportion of the communication costs, data is encoded before it is transmitted so that the communication load can be minimized. Apart from combining coding with different techniques, e.g., compression and randomization techniques to improve the performance of the shuffling phase, the coding techniques are also designed to solve different computation problems, e.g., distributed graph computation problems [108], [109] and multistage MapReduce computations [110].
- *Consideration of Underlying Network Architecture:* Generally, the communications between the workers as well as between the workers and the master are affected by the way that they are connected to each other. For example, server-rack architecture (Fig. 8) is one of the most commonly used methods to connect the various servers. By taking the underlying architecture into consideration, the effectiveness of the coding implementation in reducing communication costs can be greatly improved.
- *Function Allocation:* Similar to the allocation of files, the studies apply this approach on heterogeneous systems. In addition, some studies consider a cascaded system [111] where each Reduce function is allowed to be computed at multiple computing nodes. In some cases where the data is randomly stored at the computing nodes, e.g., when the computing nodes are constantly moving, an optimal function allocation strategy is useful in reducing the number of broadcast transmissions and thus minimizing the communication load.

A. File Allocation

The design of file allocation at each computing node is one of the major steps for the implementation of CDC scheme. There are a few approaches to an optimal file allocation strategy: (i) considering heterogeneous systems, (ii) maximizing data locality, and (iii) reducing subpacketization level.

1) *Considering Heterogeneous Systems:* As discussed in Section III-A, although the CDC scheme proposed in [17] carefully allocates the subfiles to the computing nodes in order to introduce coded multitasking opportunities, it considers a homogeneous system which may not be useful for practical implementation. In order to appropriately allocate the files to the distributed computing nodes, heterogeneous systems where the computing nodes have diverse storage, computational and communication capabilities, should be considered in determining the optimal file allocation strategy and coding scheme that minimize the communication load [112].

By leveraging on the extra storage capacity of the workers, the communication costs between the master and the workers

in the process of data shuffling are minimized. The reason is that if more data can be stored at the workers, fewer communication rounds are needed for the workers to receive shuffled data from the master. In the extreme case, if the worker can store the entire dataset, there is no communication needed for the worker to receive shuffled data in any iteration. As a result, there is a tradeoff between the storage capacity of the workers and the communication overhead in the data shuffling process. In the data shuffling process, there are two phases, namely data delivery and storage update. Instead of a random storage placement [19], a deterministic and systematic storage update strategy [113] creates more coding opportunities in transmitting data to the workers at each iteration, reducing the communication load.

2) *Maximizing Data Locality:* One of the important factors in determining the optimal file allocation strategy is data locality. Data locality is defined as the percentage of local tasks over the total number of Map tasks, i.e., the fraction of Map tasks that are allocated to the computing nodes having the required data for computations such that no communication is needed to obtain the data. High data locality means that less communication bandwidth is needed for the transmission of subfiles, which is required if the computing node does not have the needed subfiles for the execution of the Map tasks. In order to maximize data locality, the problem of allocation of Map tasks to different computing nodes can be tackled by solving a constrained integer optimization problem [114].

3) *Reducing Subpacketization Level:* As the number of computing nodes in the network increases, the input file needs to be split into a large number of subfiles. Specifically, the number of subfiles generated increases exponentially in the number of computing nodes [106]. However, there is a maximum allowable subpacketization level, i.e., number of subfiles, where the dataset can only be partitioned into a limited number of packets, beyond which the communication load increases due to more transmissions required and the unevenly-sized intermediate results. Hence, there are several reasons for the reduction of subpacketization level: (i) to reduce the communication load in the Shuffle phase even when there is a large number of computing nodes, (ii) to reduce the packet overheads which increases with the number of broadcast transmissions, and (iii) to reduce the number of unevenly-mapped outputs which require zero padding. To keep the subpacketization level below the maximum allowable level, Group-based Coded MapReduce [106] allocates the dataset based on the random groupings of the computing nodes and allows the computing nodes to cooperate in the transmission of messages.

To avoid splitting the input file too finely, the authors in [107] use an appropriate resolvable design [115], which is based on linear error correcting codes, to determine the number of subfiles, the allocation of the subfiles to the computing nodes and the construction of the coded messages in the Shuffle phase. Building on this initial work, the authors in [116] use the resolvable design based scheme to solve the limitation of the compressed CDC scheme [117] that uses both compression and coding techniques. Although the compressed CDC scheme helps to reduce communication load, it requires large number of jobs to be processed simultaneously.

Hence, the resolvable design based scheme is used to reduce the number of subfiles generated. Specifically, for each job in the compressed CDC scheme, the single-parity code is used to split the input file and the resolvable design based scheme is used to allocate the subfiles to the computing nodes. By aggregating the underlying functions and applying the resolvable design based scheme, multiple jobs can be processed in parallel while minimizing the execution time in the Shuffle phase, contributing to the reduction of the overall job execution time. Although the number of subfiles or number of jobs generated still increases exponentially with some of the system parameters, e.g., number of computing nodes and number of output functions, the exponent is much smaller when the resolvable design based scheme is implemented.

In addition to the exponential increase in the number of subfiles required, the number of output functions required also increases exponentially as the number of computing nodes in the network increases. There are other methods to reduce the number of subfiles and the number of output functions such as the hypercube computing scheme [118] and the placement delivery array (PDA) [119]–[121]. However, most of the CDC schemes consider non-cascaded systems, i.e., each Reduce function is computed at exactly one computing node [116]. In [118], a cascaded system is considered but only two values for the number of computing nodes that perform each Reduce function are considered. By applying the concept of PDA to the distributed computation framework, the performance of the proposed computation scheme is evaluated for different number of computing nodes that compute the Reduce functions [119]. Although the implementation of these various methods reduces the number of subfiles generated, it may come at the expense of higher communication load [107], [121].

B. Coded Shuffling Design

In the design of coded shuffling algorithms, we have classified the approaches into three different categories: (i) compression and randomization, (ii) coding across multiple iterations and (iii) problem-specific coding approaches.

1) *Compression and Randomization*: To further reduce the communication costs of the distributed computation tasks, the design of the coded data shuffling scheme can incorporate different techniques to create more coded multicasting opportunities. Besides, the coded shuffling schemes are designed to minimize communication costs for different distributed computation problems such as iterative algorithms, graph computations and multistage dataflow problems.

The work in [17] generates replications of the computation tasks in the Map phase in order to reduce communication load in the Shuffle phase by coding and multicasting the intermediate results. To further reduce the communication load, compression and randomization techniques can be applied to the design of the coded shuffling algorithms.

- *Compression Techniques*: The compressed CDC computation scheme is proposed in [117] by jointly using two techniques, i.e., compression and coding techniques.

Each computing node first computes the allocated Map tasks and generates the intermediate results. By using the compression techniques, several intermediate results of a single computation task are compressed into a single pre-combined value. The communication bandwidth needed to transmit a single pre-combined value is much smaller than that of transmitting several uncombined intermediate values since the size of the pre-combined value equals the size of only one intermediate value. With the pre-combined values from different computation tasks, the computing node codes them for multicasting to other computing nodes simultaneously. There are two advantages to this compressed CDC scheme: (i) the communication load is reduced proportional to the storage capacity of each computing node, and (ii) the communication load does not scale linearly, i.e., slower than linear, with the size of the dataset. In some cases, e.g., parallel stochastic gradient descent (SGD) algorithms, instead of transmitting intermediate results, computed gradient updates are exchanged among the workers. In such cases, Quantized SGD [122], a compression technique, can be used to reduce communication bandwidth used during the gradient updates between the computing nodes. In each iteration, the computing nodes are allowed to adjust the number of transmitted bits by quantizing each component to a discrete set of values and encoding these quantized gradients.

- *Randomization Techniques*: Instead of introducing coded multicasting opportunities to reduce the communication load in the Shuffle phase, there are other coding techniques that can be applied to increase efficiency of data shuffling. One of the ways is to perform a semi-random data shuffling and coding scheme based on pliable index coding which introduces randomization in the data shuffling process [123]. There are two important modifications made to the conventional pliable index coding scheme [124], which is used to minimize the number of broadcast transmissions while satisfying users' demands. Firstly, the correlation of messages between workers is reduced. In order to do so, a message should only be transmitted to a fraction of the workers so that the same message is not held by all workers. As such, the pliable index coding problem is formulated with an objective where the goal is to minimize the number of broadcast transmissions under the constraint of a maximum number of workers that can receive the same message. Secondly, the correlation of messages between iterations is reduced. The reduction of correlation of messages prevents the workers from performing computations on the same dataset after shuffling, which may be redundant. A two-layer hierarchical structure is proposed for data shuffling. In the upper layer, the messages are partitioned into multiple groups of which each group of messages is transmitted to a fraction of workers. In the lower layer, each group of messages and the corresponding allocated workers are formulated as a constrained pliable index coding problem. Randomization occurs in two stages: (i) when the master selects the

messages in each group and transmits them to the workers, and (ii) when the workers discard old messages from their cache. Experimental results show that the proposed pliable index coding requires only 12% of broadcast transmissions needed by an uncoded scheme, i.e., random shuffling with replacement scheme.

2) *Coding Across Multiple Iterations*: Most works on coded iterative algorithms focus on the optimization of a single computation iteration or the minimization of communication load in a single communication round [86]–[88], [125]. However, multiple rounds of communications are generally required to solve distributed iterative problems. In the studies of [126] and [127], the results of multiple iterations are transmitted in a single round of communication by jointly coding across several iterations of the distributed computation task. By leveraging on the computation and storage redundancy of the workers, the number of communication rounds between the master and the workers is greatly reduced, resulting in a reduction in the communication costs. However, the computation and storage costs may not be optimal as compared to uncoded computing schemes, e.g., [128] and [129], which achieve near-optimal computation and storage costs.

3) *Problem-Specific Coding Approaches*: Apart from the typical distributed computation problems, the MapReduce framework can also be used to solve distributed graph computation problems [108], [109]. However, for graph computing systems, the computation at each vertex is a function of the graph structure which means that each computation only needs data from its neighboring vertices. More specifically, the communication load in the Shuffle phase depends on the connectivity probabilities of vertices in the graph, in which each vertex is only allowed to communicate reliably with a subset of random vertices [109]. As a result, the CDC scheme proposed in [17] (which was previously discussed in Section III-A) is not applicable to solving the graph computation problems. Looking into this, the authors in [108] propose a coded scheme to solve the problem of random Erdős-Rényi graph while minimizing the communication load in the Shuffle phase. A similar inverse tradeoff curve between the computation load and average communication load is obtained by using coding techniques in solving distributed graph computations.

Moreover, many distributed computing applications consists of multiple stages of MapReduce computations. The multistage data flow can be represented by a layered DAG [110] in which the computing nodes, i.e., vertices in a particular computation stage are grouped into a single layer. Each vertex computes a user-defined function, i.e., Map or Reduce computation that transforms the given input files into intermediate results whereas the edges represent data flow between the computing nodes. By exploiting the redundancy of the computing nodes, coding techniques are applied to the computing nodes individually to minimize the communication costs. The proposed work in [110] considers a uniform resource allocation strategy where the computation of each vertex is distributed across all computing nodes. However, the communication load can be further reduced by reducing the number of computing nodes that are used to compute each

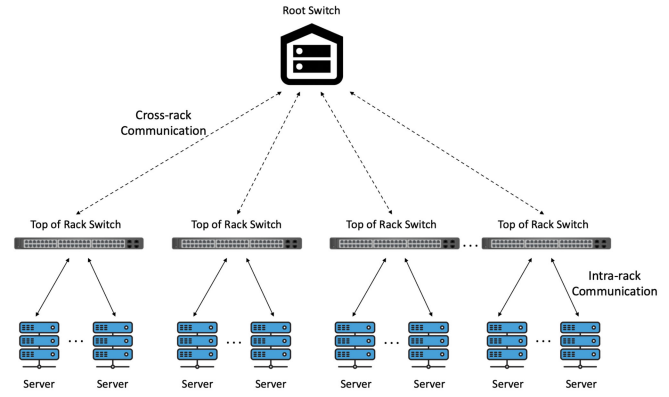


Fig. 8. Server-rack architecture where multiple servers in each rack are connected via a Top of Rack switch and the Root switch connects multiple Top of the Rack switches.

vertex. Given that fewer computing nodes are used to compute each vertex, the computation load performed by each computing node increases and the computing nodes have more local information, thus reducing the need for communication to obtain the required information. Therefore, a dynamic resource allocation strategy is needed to further minimize the communication load in the multistage MapReduce problems.

C. Consideration of Underlying Network Architecture

Although various techniques can be used jointly with the coding techniques to increase the efficiency of data shuffling phase, it is important to consider the underlying network architecture, i.e., how the servers are connected to each other, in designing the coded shuffling algorithms.

In [114], Hybrid Coded MapReduce is proposed by considering the server-rack architecture (Fig. 8) in the distributed computing systems. There are two types of communications in the Shuffle phase: (i) the cross-rack communication where the data is shuffled across different rack layers, and (ii) the intra-rack communication where the data is shuffled within the rack. In the first stage where cross-rack communication takes place, the Coded MapReduce algorithm [18] (Fig. 4) is used to create multicasting opportunities for the transmission of messages. In the second stage where the intra-rack communication is performed, data is shuffled in a unicast manner where no coding technique is applied. The simulation results show that the cross-rack communication costs incurred by the hybrid scheme are lower than those of both Coded MapReduce [18] and uncoded scheme [6] at the expense of a higher intra-rack communication costs. Although the Coded MapReduce scheme still achieves the lowest total communication costs, the overall communication costs for the hybrid scheme can be further reduced by parallelizing the intra-rack operations to provide a more accurate comparison between the different computation schemes.

The CDC computation scheme proposed in [17] is useful for networks with computing nodes that are closely located with each other and connected via a common communication bus. However, in practical distributed computing networks,

it is hard to implement a common-bus topology for the physically separated computing nodes. Hence, to reduce communication load for the distributed computation tasks, it is important to consider a practical data center network topology to reap the coding benefits of the CDC schemes. As such, in [130], the authors propose a CDC scheme based on a widely-used low-cost network topology which is the t -ary fat-tree topology [131], [132]. It has the characteristics of network symmetry and connections between any two computing nodes in the network. Given a practical network topology design, the proposed topological CDC scheme achieves optimal max-link communication load over all links in the topology of which the optimal tradeoff between the max-link communication load and the computation load is characterized.

Although the coded shuffling algorithm proposed in [19] reduces communication load of the data shuffling process, there are two main limitations: (i) it assumes a perfect broadcast channel between the master and the workers, and (ii) the theoretical guarantee of number of broadcast transmissions only holds when the number of data points approaches infinity. To overcome the limitations of the coded shuffling algorithm proposed in [19], Ubershuffle [133] fills the missing entries in the encoding tables by reallocating the data points between the workers. This reduces the number of transmitted encoded packets, resulting in a reduction of communication load. The performance of the UberShuffle algorithm is evaluated when applied to different problem settings. The experimental results show that in comparison to the coded shuffling algorithm in [19], the UberShuffle algorithm reduces the shuffling time by up to 47.2% and 35.7% when implemented with the distributed SGD algorithm for a low-rank matrix completion problem and the parallel SGD algorithm for a linear regression problem respectively.

D. Function Allocation

In most of the studies of CDC computation framework, non-cascaded systems are considered. In other words, each Reduce function can only be computed at exactly one computing node. Considering that the computing nodes have different storage, computational and communication capabilities, a non-cascaded heterogeneous system [134] where each Reduce function is computed exactly once is considered. In this proposed scheme, the computing nodes are allocated with different number of Reduce functions of which the computing nodes with greater storage and computational capabilities are allocated more Reduce functions. This heterogeneous Reduce function allocation creates a symmetry among the multicast groups such that each computing node in a group requests the same number of intermediate outputs from the other computing nodes in the same group. The heterogeneous CDC system achieves a lower communication load than an equivalent homogeneous CDC system.

However, it is desirable for the Reduce functions to be computed at multiple computing nodes in some applications, e.g., iterative algorithms in the Spark model [49] where the output of a MapReduce procedure acts as the input to the MapReduce procedure in the next iteration. Although the work in [17]

generalizes the CDC framework by allowing each Reduce function to be computed at more than one computing node, it only applies to homogeneous systems. Similar to [134], heterogeneities of the systems are also considered in [111]. However, instead of a non-cascaded system, the authors propose a more general framework of cascaded CDC [111] on heterogeneous systems. In other words, each Reduce function is allowed to be computed at multiple computing nodes. Since the computing nodes have different storage capacities, the number of subfiles stored at each computing node differs. The computing nodes with larger storage capacities are allocated more subfiles and thus, compute more intermediate results. Instead of allocating the files and functions randomly as in the work in [17], this cascaded CDC scheme uses a hypercube approach [118] to allocate the files and functions to the computing nodes. The simulation results show that for the same number of computing nodes in the network, the proposed cascaded CDC scheme achieves smaller communication load than the state-of-art schemes that consider homogeneous systems [17], [118].

In the study of [135], the allocation of the functions to the computing nodes does not depend on their capabilities but on the data stored at the nodes, of which the data placement is assumed to be random. This is very useful for applications that the computing nodes are mobile and collect data on-the-move. If the probability that the computing nodes contain data is higher than the pre-defined threshold, it is possible to allocate Reduce functions such that the computing nodes do not need to exchange their intermediate results for the computations of Reduce functions, i.e., each computing node can compute the Reduce functions based on its locally stored data. This reduces the number of broadcast transmissions in the Shuffle phase, thus minimizing the communication load.

Although heterogeneities of the systems are taken into consideration in some of the works [111], [112], [134], they merely focus on either file allocation or function allocation. On one hand, the work in [112] proposes an optimal file allocation strategy in consideration of the heterogeneous storage capabilities of the computing nodes. However, the Reduce functions are distributed uniformly among the computing nodes. On the other hand, the works in [111] and [134] propose the allocation of Reduce functions based on the computational capabilities of the computing nodes. However, the input file is split equally and distributed among the computing nodes. Considering a more generalized heterogeneous system, a joint file and function allocation strategy is proposed in [136] to reduce the communication load in the Shuffle phase. The file allocation and function assignment strategies allocate more subfiles and Reduce functions respectively to the computing nodes with higher computational and storage capabilities. Generally, the Reduce function assignment is related to the input file allocation as the computing nodes with more input files have greater storage and computational capabilities and hence are more capable of computing more output functions. In particular, there are two proposed schemes of function assignment, i.e., computation-aware function assignment and shuffle-aware function assignment. For computation-aware function assignment strategy, the number

of functions allocated is proportional to the computational capabilities of the computing nodes in order to reduce computation latency. For the shuffle-aware function assignment strategy, the functions are mostly allocated to the computing nodes with high computational capabilities so that the communication load in the Shuffle phase is minimized. The simulation results show that the communication loads achieved by both computation-aware and shuffle-aware function assignment strategies are lower than the uniform function allocation strategy. Besides, the computation-aware function assignment strategy requires fewer number of output functions as compared to the proposed schemes in [111] and [134]. However, the number of input files required is much larger, especially when the number of computing nodes in the system increases.

While there has been great attention on the design of file allocation, coded shuffling algorithms and function allocation, all these works assume a fixed number of computing nodes in the distributed computing systems. For a given computation task that specifies the number of subfiles and the number of outputs, the allocation of functions and the data shuffling schemes can be implemented with minimum number of computing nodes. In the study of [137], the resource allocation problem is formulated as an optimization problem that minimizes the overall job execution time with optimal number of computing nodes used. For more practical implementation, the resource allocation strategy should consider the heterogeneity in the processing speed of the nodes, since the straggler effects cause an increase in computation latency which increases the overall job execution time.

By exploiting the fact that the computing nodes have time-varying computing resources, e.g., the computing nodes may have different central processing unit (CPU) power over time, an optimal computation task scheduling scheme helps to reduce the communication load. In the scheduling of tasks under dynamic computing resources, the total communication load is minimized by optimizing the number of allocated tasks and load redundancy at each time slot [138].

E. Summary and Lessons Learned

In this section, we have reviewed four approaches to minimize communication costs in distributed computing systems. For each approach, we discuss the solutions that are proposed in different works. We summarize the coding schemes to minimize communication costs in Table VI. The lessons learned are as follows.

- To handle the increasing amounts of data generated, more computing nodes are needed for the completion of distributed computations given the limited capabilities of the computing nodes. With more computing nodes connected to the network, more communication rounds of the computed intermediate results are required, resulting in higher communication costs which lead to a longer job execution time. Besides, the high communication costs in the data shuffling phase impede the implementation of efficient distributed iterative algorithms which are useful for the training of machine learning models. As such, the minimization of communication costs is a key step to

achieve the objective of reducing the overall job execution time of the distributed computation tasks.

- While having more computing nodes to perform the computation tasks in parallel helps to reduce the computation load of each computing node, the communication costs may increase, slowing down the entire computation process. Instead of generating infinitely large number of subfiles and distributing them among large number of computing nodes, several studies focus on determining the optimal number of subfiles to avoid the input file from splitting too finely. For example, the resolvable design based schemes [107], [116] and PDA approaches [119]–[121] are adopted to split the input files. Hence, it may not be optimal to use all the computing nodes that are connected to the network. In fact, the authors in [137] propose an optimal resource allocation scheme that determines the minimum number of computing nodes needed to achieve the minimum overall job execution time.
- Coding techniques have shown to be effective in reducing communication load in the data shuffling phase at the expense of higher computation load [17], [19]. However, the two-dimensional tradeoff is insufficient to fully evaluate the performance of the CDC schemes. Apart from leveraging on the computational capabilities of the computing nodes, their storage capabilities can be exploited. For example, more data can be stored at computing nodes with higher storage capacities such that the number of communication rounds is reduced [113]. Moreover, by considering the data stored at the computing nodes, the allocation of functions that maximizes data locality helps to reduce the need for communication bandwidth [114]. Hence, instead of the two-dimensional tradeoff between computation load and communication load, the three-dimensional tradeoff between computation, communication and storage cost [74] has to be carefully managed for the implementation of efficient CDC schemes.
- For effective implementation of CDC schemes in practical distributed computing systems, the underlying architecture has to be considered. Generally, the distributed computing systems operate under the server architecture which consists of multiple racks where each rack has several servers. The Hybrid Coded MapReduce [114] scheme reduces cross-rack communication at the expense of higher intra-rack communication. Besides, the communication channels between the master and workers are imperfect. As a result, the theoretical gains of the coded data shuffling schemes are not achievable under practical setups. In order to design and analyze the performance of the CDC schemes, the limitations of the distributed computing systems should be taken into consideration.
- Most of the CDC schemes focus on the minimization of communication costs in the data shuffling phase at the expense of reasonable increase in computation load. However, the computational overhead of the algorithm is not negligible under some settings. For example, under much faster broadcast environment, the UberShuffle algorithm [133] incurs significant encoding and decoding

TABLE VI
CDC SCHEMES TO REDUCE COMMUNICATION COSTS

Approach	Ref.	Coding Schemes	Key Ideas	Platform Support
File Allocation	[113]	-	Deterministic and systematic storage update strategy	Heterogeneous
	[114]	Hybrid Coded MapReduce	Allocates Map tasks such that data locality is maximized	Homogeneous
	[106]	Group-based Coded MapReduce	Allocates dataset by using a group-based method in order to avoid high subpacketization level and allow computing nodes to cooperate in the transmission of messages	Homogeneous
	[107]	Resolvable Design	Uses single-parity code to determine the number of subfiles and allocate the subfiles based on the corresponding resolvable design	Homogeneous
	[119]–[121]	Placement Delivery Arrays	Construction of CDC schemes based on PDA which has the property of illustrating the placement and delivery phase in a single array	Homogeneous
Coded Shuffling Design	[117]	Compressed CDC	Pre-combines computed intermediate values of the same function, followed by coding the pre-combined packets for communication between different computing nodes	Homogeneous
	[122]	Quantized SGD	Quantizes the component of the gradient vector to a discrete set of values and encodes the quantized gradients given their statistical properties	Homogeneous
	[123]	Pliable Index Coding	Semi-random data shuffling scheme based on modified pliable index coding to reduce the number of communication rounds	Homogeneous
	[126]	Cross-iteration Coded Computing	Jointly codes across multiple iterations for a single communication round	Homogeneous
	[108]	CDC for distributed graph computing systems	Instead of communicating with all other computing nodes, each computing nodes only needs to communicate with a subset of computing nodes to obtain the required data to complete its computation tasks	Homogeneous
	[110]	CDC for multistage dataflow	A more generalized CDC scheme is proposed to handle multistage dataflow computation tasks which are represented by layered DAGs	Homogeneous
Consideration of Underlying Network Architecture	[114]	Hybrid Coded MapReduce	Reduces cross-rack communication at the expense of higher intra-rack communication based on the server-rack architecture	Homogeneous
	[130]	Topological CDC	Considers t-ary fat-tree topology which is a more practical topology to connect the physically separated computing nodes in data center networks	Homogeneous
	[133]	UberShuffle	Considers imperfect communication channel between the workers and the master	Homogeneous
	[139]	-	Considers multicore setup where each computing node can have multiple cores, e.g., the CPU instances of publicly available cloud infrastructure can deliver up to 128 cores	Homogeneous
Function Allocation	[134]	-	Considers a non-cascaded system and allocates Reduce functions over a simplified heterogeneous network which comprises multiple homogeneous networks	Heterogeneous
	[111]	Cascaded CDC	Reduce functions are computed at more than one computing node and they are allocated based on the combinatorial design in [118]	Heterogeneous
	[135]	-	Allocates functions to maximize data locality such that the number of communication rounds required is reduced	Homogeneous
	[136]	-	Joint file and function allocation strategy	Heterogeneous
	[138]	-	Considers the availability of time-varying computing resources	Homogeneous

costs such that the shuffling gain is offset by the high computational overhead. For future works, more practical CDC schemes can be proposed such that the communication costs are minimized while maintaining low computational cost to improve the performance of the distributed computing systems. Given the uncoded computing schemes, e.g., [128] and [129], achieve near-optimal computation and storage costs, one possible research direction is to merge the communication-efficient proposed schemes with the uncoded computing schemes to reduce the computation and storage costs.

- Although some of the works consider heterogeneities in the capabilities of the computing nodes to allocate files and functions, the presence of stragglers which have slower processing speeds still hinder the performance of the distributed computing systems. Therefore, we further discuss the approaches to mitigate the straggler effects in the next section.

V. MITIGATION OF STRAGGLERS

The problem of straggler effects has been widely observed in the distributed computing systems. Previously, various

methods such as straggler detection [68], [140], asynchronous execution and naive replication of jobs [14], [71] have been proposed to reduce the overall time taken to execute the computation tasks. Recently, coding approaches have been shown to outperform the aforementioned methods in reducing computation latency of the distributed computing systems. The authors in [141] have discussed in-depth the fundamental limits of different coded computation strategies that provide resiliency against unreliable components in the networks. In this section, we discuss three approaches to mitigate the straggler effects.

- *Computation Load Allocations:* Coding techniques can be implemented together with computation load allocation strategies to reduce the computation latency in the distributed computing systems. It is important to take into account the variation in the computational capabilities of the computing nodes to allocate computation load. As such, different prediction methods such as an long short-term memory (LSTM) algorithm [142], an Auto Regressive Integrated Moving Average (ARIMA) model and a Markov model [143] are used to estimate the processing speeds of the nodes. Generally, the objective of the load allocation strategies is to minimize computation latency. However, in some applications where strict deadlines are given, the load allocation strategies aim to maximize timely computation throughput [143].
- *Approximate Coding:* For some applications, e.g., location-based recommendation systems, exact solutions are not necessary. The studies explore different coding approaches to obtain approximate solutions to the problems. The approximate coding methods relax the requirement for convergence and thus reduce the number of workers that are required to return their computed results. This can avoid stragglers to make an adverse effect to the system.
- *Exploitation of Stragglers:* The stragglers may have completed a fraction of the allocated computation tasks, of which is a waste to be ignored completely. In fact, the stragglers may not be persistent over the entire computation process [144] and thus their partial computed results can be useful to recover the final result. In order to maximize the resource utilization of the stragglers, the workers are allowed to sequentially process their allocated subtasks and transmit their partial computed results continuously [145]. However, this may come at the expense of higher communication load, which needs to be carefully managed.

A. Computation Load Allocation

Apart from having different straggling rates that affect the completion of tasks, the computing nodes have different capabilities, e.g., storage capacities, computing resources and communication bandwidth. To better handle the stragglers in the distributed computing systems, an optimal load allocation strategy that takes into account these heterogeneities is necessary to minimize the overall job execution time. Given the computation time parameters, i.e., the straggling and shift

parameters of each worker, the Heterogeneous Coded Matrix Multiplication (HCMM) algorithm [146] determines the allocation of computation load to each worker. The HCMM scheme exploits the benefits of both coding techniques and computation load allocation strategy to minimize the average computation time of the computation tasks. Given that it is difficult to derive closed-form expressions of expected computation time of the heterogeneous computing nodes, a two-step alternative problem formulation is proposed. In the first step, given a time period, the number of computed results by the workers is maximized by optimizing the load allocation. In the second step, given the derived load allocation in the first step, the time needed to ensure sufficient results are returned at a pre-defined probability is minimized. The simulation results show that when workers' computation time is assumed to follow a shifted exponential runtime distribution, HCMM reduces the average computation time by up to 71%, 53% and 39% over uniform uncoded, load-balanced uncoded and uniform coded load allocation schemes, respectively. In practical experiments over Amazon EC2 clusters, the combination of HCMM and LT codes outperforms the uniform uncoded, load-balanced uncoded and uniform coded load allocation schemes by up to 61%, 46% and 36%, respectively.

Although HCMM achieves asymptotically optimal computation time, the decoding complexity is high, which suggests the opportunity to further speed up the overall computation tasks. In practical distributed computing systems, some computing nodes have the same computational capabilities, in terms of the same distributions of computation time, and thus they can be grouped together. By exploiting the group structure and heterogeneities among different groups of computing nodes [147], [148], the implementation of a combination of group codes and an optimal load allocation strategy not only approaches the optimal computation time that is achieved by the MDS codes, but also has low decoding complexity. In addition, by varying the number of allocated rows of the matrix to the workers [148], the computation latency can be reduced by orders of magnitude over the MDS codes with fixed computation load allocation [147] as the number of workers increases. The load allocation strategy proposed in [148] focuses mainly on the design of an optimal MDS code. Other efficient coding algorithms such as LT codes [79] can also be explored in future studies.

In addition to the heterogeneous capabilities of the computing nodes, the amount of available resources of the computing nodes may vary over time. By always allocating computation tasks to the computing nodes with higher capabilities, delays may be incurred in completing the allocated tasks if the computing nodes start to work on the newly-allocated computation tasks in parallel. At the same time, the resources of the computing nodes with lower capabilities may be under-utilized. In order to maximize the resource utilization of the computing nodes, dynamic workload allocation algorithms which are adaptive to the time-varying capabilities of the computing nodes are proposed [142], [143], [149]. To provide robustness against the stragglers, the design of the load allocation algorithms often depends on the historical data of the computing nodes such as computation time, which can be used to predict

the processing speeds by using an LSTM algorithm [142], an ARIMA model or a Markov model [143]. The best performing LSTM model achieves 5% lower prediction error than an ARIMA (1,0,0) model [142].

In the study of [149], the authors propose Coded Cooperative Computation Protocol (C3P) which is a dynamic and adaptive coded cooperation framework that efficiently utilizes the available resources of the computing node while minimizing the overall job execution time. Specifically, the master determines the coded packet transmission intervals based on the responsiveness of the computing nodes. For the computing nodes which are not able to complete the tasks within the given transmission interval, they suffer from longer waiting time for the next coded packets. In comparison to the HCMM scheme [146] which does not consider the dynamic resource heterogeneity in workers, the C3P framework achieves more than 30% improvement in task completion delay.

The dynamic and adaptive load allocation algorithms are especially useful in providing timely services with deadline constraints which are common in many IoT applications. For such applications, instead of minimizing task completion delay, the objective of the load allocation algorithms is to maximize timely computation throughput, i.e., the average number of computation tasks that are successfully completed before the given deadline [143], [150].

For some applications that may need timely but not necessarily optimal results, it is more important to recover the final result with the highest accuracy possible by the stipulated deadlines than to solve for an exact solution. The algorithm to solve for an approximate solution requires significantly shorter computation time than that of an algorithm that solves for an exact solution [151]. In the study of [152], the batch-processing based coded computing (BPCC) algorithm is proposed. The workers first partition the allocated encoded matrix into several batches, i.e., submatrices, for computations. As soon as the workers complete the computation of each batch of submatrix with a given vector, they return the partial computation results which are used to generate the approximate solution. Based on the computation time parameters, i.e., the straggling and shift parameters of the workers, the BPCC algorithm is used to optimally allocate the computation load to each worker by considering two cases: (i) negligible batching overheads, and (ii) linear batching overheads. Hence, the allocation of computation load is optimized by jointly minimizing both the overall job execution time as well as the potential overhead of batch processing. In addition to reducing computation latency, the BPCC algorithm exploits the partial computation results of all computing nodes, including the stragglers, which contribute to approximate solutions of higher accuracy [145]. The simulation results show that BPCC algorithm with negligible batching overheads achieves up to 73%, 56% and 34% reduction in average job execution time over uniform coded, load-balanced uncoded and HCMM [146] load allocation schemes respectively. The experimental results on Amazon EC2 clusters and Unmanned Aerial Vehicles (UAVs) based airborne computing platform also demonstrate similar results.

B. Approximate Coding

For some applications, it is not necessary to obtain exact solutions. In this subsection, we review the coding approaches that are used to derive approximate solutions for the distributed computation tasks: (i) matrix multiplication, (ii) gradient descent, (iii) non-linear computations. Table VII summarizes the approximate coding schemes for various distributed computation tasks.

1) *Matrix Multiplications*: The anytime coding [151] scheme derives an approximate solution by using the output results of the completed computing nodes at any given time. Based on singular value decomposition (SVD), the given computation task is decomposed into several subtasks of different priorities. More important subtasks are allocated more computing nodes for computations as they improve the accuracy of the approximation. To allow the users to receive useful information from time to time, approximate solutions can be transmitted to the users sequentially. This can be achieved by solving a sequence of approximated problems [153], instead of solving the original problem directly.

To further reduce the computation time of approximate matrix multiplications, sketching techniques [161], [162] can be used to remove redundancy in the structure of the matrices through dimensionality reduction. However, by using sketching techniques, the recovery threshold increases as the redundancy is removed. In contrast, coding techniques reduces recovering threshold by introducing redundancy. As such, a combination of both techniques that carefully manage the tradeoff between the recovery threshold and the amount of redundancy can be implemented to minimize computation latency. In particular, count-sketch technique [163] is combined with structured codes to mitigate the straggler effects by preserving a certain amount of redundancy, thereby achieving the optimal recovery threshold and hence computation latency [154], [155].

2) *Gradient Descent*: To speed up the distributed gradient descent tasks, several approximate gradient coding schemes are proposed to approximately compute any sum of functions. Instead of constructing the gradient codes based on expander graphs [87] which are difficult to compute due to high complexity, a more efficient and simpler Bernoulli Gradient Code (BGC) is proposed by using sparse random graphs [125] which introduce randomness into the entries of the function assignment matrix. Since the performance of the gradient codes depends on the efficiency of the decoding algorithms, the authors also present two decoding techniques to recover the approximate solution from the outputs of the non-stragglers. The simulation results show that the BGC schemes can handle adversaries with polynomial-time computations but at a cost of higher decoding error than the FRC schemes [86]. Besides, the optimal decoding algorithm always achieves a lower decoding error than that of the one-step decoding algorithm across various gradient coding schemes. A rigorous convergence analysis of the approximate gradient codes and the performance of BGC on different practical datasets such as the Amazon dataset, Covertype dataset and KC Housing dataset are presented in [164]. Stochastic block

TABLE VII
APPROXIMATE CDC SCHEMES TO MITIGATE THE STRAGGLER EFFECTS

Problems	Ref.	Coding Schemes	Key Ideas
Matrix-Vector	[151]	Anytime Coding	Computations can be stopped anytime and the approximate solution is derived from the computing nodes that have completed their tasks
	[153]	Coded Sequential Computation Scheme	A sequence of approximated problems are designed such that the time required to solve these problems is shorter than solving the original problem directly
Matrix-Matrix	[154]	CodedSketch	Use a combination of count-sketch technique and structured polynomial codes
	[155]	OverSketch	Divide the sketched matrices into blocks for computations
Gradient Descent	[125]	Bernoulli Gradient Codes	Use Bernoulli random variables as entries of the function assignment matrix
	[156]	Stochastic Block Codes	Interpolation between BGC and FRC
	[157]	Stochastic Gradient Coding	Distribute data based on pair-wise balanced scheme and provide a rigorous convergence analysis of the proposed coding scheme
	[15]	LDPC Codes	Encode the second moment of the data points
	[158], [159]	Encoded Optimization	Encode both the labels and data such that the redundancy is introduced in the formulation of the optimization problems
Non-Linear	[101]	Generalized PolyDot	Generalization of PolyDot codes [84] and is used for the training of DNNs
	[160]	Learning-based Approach	Design neural network architectures to learn and train the encoding and decoding functions to approximate unavailable outputs

code [156] which is based on the stochastic block model from random graph theory, is an interpolation between FRC [86] and BGC [125]. On one hand, the FRC schemes achieve small reconstruction errors under random straggler selection while on the other hand, the BGC schemes are robust against polynomial-time adversarial stragglers.

Other approximate gradient coding methods such as the Stochastic Gradient Coding (SGC) [157] and LDPC codes [15] are used to obtain an estimate of the true gradient. Similar to the idea of encoding data batches in the PCR schemes [90], the data variables in the optimization formulation can be efficiently encoded to mitigate the straggler effects in more general large-scale optimization problems such as support vector machines, linear regressions and compressed sensing [158], [159]. Generally, in solving for approximate solutions to reduce computation latency of the distributed tasks, there is an inherent tradeoff between the recovery threshold and the approximation error where the recovery threshold can be reduced at the expense of larger approximation error [165].

3) *Non-Linear Computations*: By leveraging on the enormous amounts of data generated, machine learning algorithms are useful in making predictions and allowing devices to respond intuitively to user demands without human interception. Different neural network architectures are developed to make accurate inference given the dataset. Since some of the layers of the neural networks such as the max-pooling functions and the activation layer are non-linear, the overall computation of the functions are non-linear. As a result, most of the prior works on linear computations which are discussed in Sections III-B, V-B1, and V-B2, are not applicable to the computation of the increasingly important non-linear neural networks, in which their performances are also limited by the stragglers. One of the few approaches that can be extended to the training of DNNs is the Generalized PolyDot codes [101] which are used to compute matrix-vector multiplications. The Generalized PolyDot codes are used to code the linear operations at each layer of the neural networks. This coding scheme

allows for errors in the training of each layer. In other words, decoding can still be performed correctly given that the amount of errors does not exceed the maximum error tolerance level. The effectiveness of coding techniques in mitigating the straggler effects of different neural network architectures such as AlexNet [12] and Visual Geometry Group (VGG) [166] in an IoT system is illustrated in [167]. However, this unified coded DNN training strategy may not be relevant to the training of other neural networks which have large number of non-linear functions. As such, the authors in [160] propose a learning-based approach for designing codes. Based on the dilated CNN and multilayer perceptrons (MLP), neural network architectures and a training methodology are proposed to learn the encoding and decoding functions. The outputs of the decoding functions are used to approximate the unavailable outputs of any differentiable non-linear function. The simulation results show that the learning-based approach to designing the encoding and decoding functions can accurately reconstruct 95.71%, 82.77% and 60.74% of the unavailable ResNet-18 classifier outputs on MNIST, Fashion-MNIST and CIFAR-10 datasets respectively.

C. Exploitation of Stragglers

To avoid delays caused by the stragglers in the network, most distributed computation schemes ignore the work completed by the stragglers by either increasing the workload of the non-stragglers or by obtaining less accurate approximate solutions. However, the amount of work that has been completed by the stragglers, especially the non-persistent stragglers, is non-negligible and can be better utilized.

In order to exploit the computational capacities of these non-persistent stragglers, multi-message communication (MMC) is used where the workers are allowed to send multiple messages to the master at each iteration. This allows the workers to transmit their partial computed results whenever they complete a fraction of the allocated task, rather than completing the entire computation task before transmitting the computed result in

a single communication round. The work in [144] considers the implementation of Lagrange Coded Computing (LCC) with MMC to minimize the average job execution time at the expense of higher communication load due to the increase in number of messages transmitted by the workers to the master. Since the LCC scheme utilizes polynomial interpolation to recover the final result, the decoding complexity and the number of transmissions can be further reduced by increasing the number of polynomials used in decoding the computed results returned by the workers. The simulation results show that by exploiting the computing resources of the non-persistent stragglers via MMC, the average job execution time decreases as the computation load of each worker increases. However, since the communication load of the LCC-MMC scheme is constant as the computation load increases, it is suitable to be implemented only when computation time dominates the overall execution time of the distributed tasks. The total time needed to execute the distributed tasks includes the time needed for both computation and communication. Otherwise, if communication load is the cause of the bottleneck of the network, LCC without MMC should be used instead since the communication load can be reduced at the expense of higher computation load.

Given that MMC is allowed where the workers perform more than one round of communication for each iteration, the computation work done by the stragglers can be exploited by allowing sequential processing where the workers need to transmit the computation results of their completed sub-task before working on the next subtask. To fully exploit the useful information provided by the stragglers, the hierarchical coded computation scheme is proposed in [145] to utilize the computations from all workers. Each worker first divides the allocated computation task into layers of sub-computations, which are processed sequentially, i.e., the result of a layer of sub-computation is transmitted to the master before the computation of the next layer starts. Since the workers have different processing speeds and the sub-computations are performed sequentially, the finishing time for each layer is different. MDS codes are used to encode the layers so that the finishing time of each layer is approximately the same. The top layers which have lower probability of erasure are encoded with higher-rate MDS codes whereas the bottom layers are encoded with lower-rate MDS codes. The simulation results show that for the same amount of tasks to be completed, the proposed hierarchical coded computation scheme achieves 1.5 factor improvement in expected finishing time as compared to the coded computation scheme proposed in [19] which ignores the computations of the stragglers.

In the study of [168], by computing the block products sequentially, the partial computation results from the stragglers are used to aggregate the final result. In order to preserve the sparsity of matrices for processing by the workers, instead of coding the entire matrix, the fraction of coded blocks can be specified to control the level to which coding is utilized in the solutions. There are two different approaches considered, depending on the placement of the coded blocks. When the coded blocks appear at the bottom of the workers, it is easier for the master to decode. When the coded blocks appear at

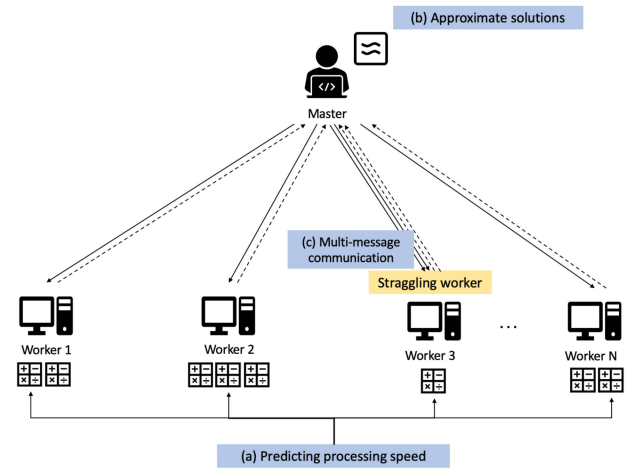


Fig. 9. Approaches to mitigate the straggler effects include (a) computation load allocation by predicting the speed of the computing nodes [142], [143], (b) approximate coding and (c) exploitation of stragglers by allowing multi-message communications [144], [145].

the top of the nodes, it minimizes the computation load of the workers. As such, the placement can be calibrated based on the task requirements.

While the coded computation schemes achieve low communication load and reduce the average job execution time for each iteration, the uncoded computation schemes have their own benefits of having no decoding complexity and allowing partial gradient updates. In order for a system to benefit from the advantages of both schemes, coded partial gradient computation (CPGC) scheme is proposed in [169]. In the CPGC scheme, uncoded submatrices are allocated for the first computation since there is a high probability for each worker to complete their first computation task. Subsequently, coded submatrices are allocated to handle the stragglers. The master is able to update the gradient parameters by using the computation results from a subset of workers and by exploiting the partial computations of the stragglers. As a result, the average job execution time for each iteration is reduced. A similar CDC scheme is proposed in [170] to solve matrix-vector multiplication problems.

D. Summary and Lessons Learned

In this section, we have discussed three approaches (Fig. 9) to mitigate the straggler effects. The lessons learned are as follows.

- The straggler effect is a key issue to be resolved in order to reduce computation latency, hence minimizing the overall job execution time. Due to various factors such as insufficient power, contention of shared resources, imbalance work allocation and network congestions [67], [68], some computing nodes may run slower than the average or even be disconnected from the network. Since the computation tasks are only completed when all computing nodes complete their computations, the time needed to complete the tasks is determined by the slowest computing node. Coding techniques have shown their

effectiveness in reducing computation latency by introducing redundancy [19]. In this section, we have explored the use of coding techniques for different distributed computation tasks, e.g., matrix-vector multiplications, matrix-matrix multiplications, linear inverse problems, iterative algorithms, convolutions and non-linear problems. While most of the research focuses on the design of encoding techniques, the decoding complexity of the codes also affects the computation latency significantly. Apart from Reed-Solomon codes [88] and LDPC codes [15], more effective codes with low decoding complexity can be investigated in future studies.

- Considering heterogeneities in the capabilities of the computing nodes, effective computation load allocation strategies are implemented to allocate workload to the computing nodes. We have discussed the proposed computation load allocation algorithms under different constraints, e.g., strict deadlines and time-varying computing resources. In addition, different prediction methods such as the LSTM algorithm [142], an ARIMA model and a Markov model [143] that predict the speeds of the computing nodes are explored. However, the stragglers may be non-persistent in nature and thus they may be useful when they are able to perform computations faster than the average rate. Hence, the load allocation based on the responsiveness of the computing nodes may be more useful in such situations.
- Instead of exact solutions, it is acceptable to derive approximate solutions for some applications, e.g., location-based recommender systems. Various coding techniques to derive approximate solutions are investigated. For example, in the studies of [154] and [155], sketching techniques are used with structured codes to minimize computation latency. However, there exists a tradeoff between the recovery threshold and the approximation error. For future works, an improvement to this tradeoff can be investigated.
- Although the stragglers run slower than the average, the computations that are completed may still be useful. It is wasteful to not utilize the partial computed results of the stragglers. Besides, these partial computed results can help to improve the accuracy of the estimates. For example, in [171], the stragglers are treated as soft errors instead of erasures to minimize the mean-squared error of the iterative linear inverse solvers under a deadline constraint by using approximate weights. Outputs from all computing nodes, including the stragglers are used to recover estimates that are as close as possible to the convergence values when the computation deadline is brought forward or when the number of computing nodes increases. Unfortunately, as the computing nodes are required to send their partial results once they complete, more communication rounds are performed. The high communication costs may be the bottleneck of the distributed computation tasks. Given the advantages of using partial results from the stragglers, optimization approaches to minimize the communication costs between the master and the workers should be explored.

- The current studies in this section have proposed effective coding schemes for implementation. However, they do not consider security in the design of the coding schemes. For example, the FRC scheme [86] achieves high accuracy in the presence of stragglers, but it is susceptible to attacks from adversarial stragglers, which turn more computing nodes into stragglers. Besides, other than the straggling workers, there may exist malicious or curious workers that may compromise the privacy and security of the system. Therefore, approaches to ensure secure coding are discussed in-depth in the next section.

VI. SECURE CODING FOR DISTRIBUTED COMPUTING

In distributed computing, the data owner, master, and workers may not belong to the same entity. For example, the data owner may wish to perform a task on a massive dataset on which intensive computations have to be performed. The computations may be divided and distributed to multiple workers on third-party computing services. However, sensitive data, e.g., in healthcare services [172], may be involved. In this case, *curious* workers may collude to obtain information about the raw data, whereas *malicious* workers [173] may intentionally contribute erroneous inputs to introduce biases to the model. Besides, in some cases, the dataset does not belong to either the master or the workers, and as such, the raw dataset has to be guarded against both parties.

To ensure that privacy and security are preserved during the computing tasks, conventional methods such as homomorphic encryption [174] have been proposed in which the data is first encrypted before being shared to workers. However, the encryption techniques are found to be costly in terms of computation and storage costs [175]. Besides, the secure multi-party computation (MPC) approaches [176]–[178] mainly focus on the correctness and privacy of the data [20], whereas neglecting to reduce the complexity of computation at each workers, or the number of workers required to perform the task. Recently, coding techniques that have originally been introduced to mitigate straggling workers are increasingly utilized to provide information-theoretic privacy guarantees. Specifically, information-theoretic privacy considers the setup consisting honest but curious workers, in which collusions formed between T of N workers do not leak information about the training dataset. Coding schemes can be used to not only mitigate the stragglers but also the colluding curious workers and malicious workers as illustrated in Fig. 10. For ease of exposition, we classify the related studies into two main categories in this section.

- *Secure Distributed Computing*: In this subsection, the studies aim to reduce the number of workers needed for information-theoretic privacy, i.e., where the colluding workers are unable to infer sensitive information from the dataset. For some studies, this objective is met while simultaneously preserving the efficiency of distributed computing, e.g., through providing resiliency against straggling workers [21].
- *Secure Distributed Matrix Multiplication (SDMM)*: The studies in the aforementioned category mainly focus on

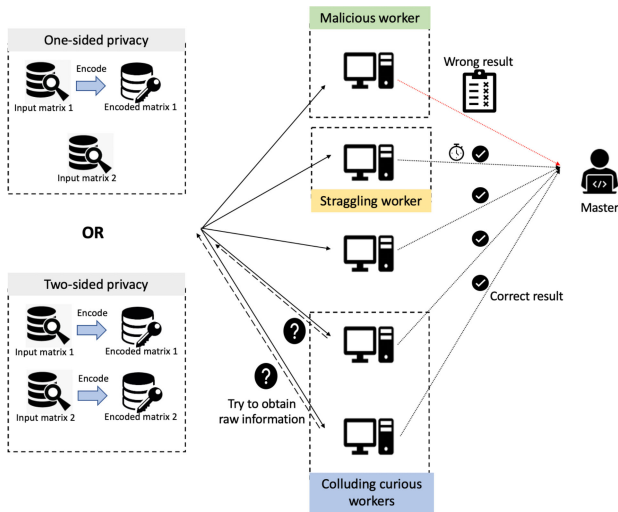


Fig. 10. Illustration of objective of the CDC schemes to preserve security and privacy by mitigating three types of workers, i.e., stragglers (produce correct results with delays), colluding curious workers (produce correct results but aim to retrieve sensitive information), and malicious workers (produce wrong results to corrupt the model).

generic operations, e.g., addition, subtraction, multiplication, or computation of polynomial functions, whereas the studies in this category focus specifically on matrix multiplication. One key difference between the two categories is that SDMM considers the specific scenario in which *both* input matrices in the multiplication operation are private information, i.e., two-sided privacy [179], whereas the prior category mainly considers one-sided privacy, i.e., only one input matrix is private. In addition, a performance metric of interest in the SDMM literature is the download rate, i.e., the ratio of the size of desired result to the total amount of information downloaded by a user from the workers.

A. Secure Distributed Computing

In Section III-B, we discuss that the polynomial codes proposed in [83] have the desirable property of an optimal recovery threshold that does not scale with the number of workers involved. In consideration of this useful property, the authors in [20] propose the polynomial sharing approach which combines the polynomial codes and the Ben-Or, Goldwasser, and Wigderson (BGW) scheme [180]. The system model considered in this study is that the data originates from external sources, and thus has to be kept private against both the workers and master. In contrast to the BGW approach which uses Shamir's scheme to encode the dataset, the study of [20] proposes to encode the dataset using the polynomial coding scheme. The authors show that the polynomial sharing approach may be applied to perform several procedures, e.g., addition, multiplication, and the computation of polynomial functions, while reducing the number of workers required to complete the task as compared to conventional MPC approaches even when workers have capacity-limited communication links.

TABLE VIII
COMPARISON OF BGW, LCC, AND HARMONIC CODING SCHEMES FOR GRADIENT-TYPE COMPUTATIONS [21], [180], [185]

	BGW	LCC	Harmonic
Min. number of workers	$K(\deg g + 1)$	$K \deg g + 1$	$K(\deg g - 1) + 2$

Typically, in conventional polynomial coding schemes, the dataset on which computations are performed is divided into multiple sub-tasks, with one sub-task encoded and assigned to each worker. In this case, faster workers that complete their task will be idle while waiting for straggling workers. To further mitigate the straggler effects, the authors in [181] leverage on computation redundancy to propose the private asynchronous polynomial coding scheme in which a computation task is divided into several relatively smaller sub-tasks for distribution to each worker. This results in two key advantages, in addition to retaining the privacy preservation properties of polynomial coding. Firstly, the smaller sub-tasks can be successfully completed by straggling workers with limited computing capacity. Secondly, workers of the fastest groups are assigned more tasks to continue working throughout the whole duration rather than wait for the stragglers, thus reducing the computation time.

However, the studies [20], [181] mainly utilize polynomial coding for privacy preservation which is restrictive in certain aspects, e.g., it only allows column-wise partitioning of the matrices [99]. As such, the entangled polynomial codes [99] are applied by [182] as an extension to polynomial sharing, so as to further reduce the restrictions during the data sharing phase, and hence, the number of workers required to perform the same computations while meeting privacy constraints.

While the studies of [20], [182] consider the scenario in which honest-but-curious workers are involved, workers may randomly be malicious in nature. As an illustration, a group of workers may be involved to compute gradients towards training a machine learning model. However, the gradients may be intentionally misreported by the workers to introduce biases or inaccuracies to the model [173]. An existing approach is to perform median, rather than mean, based aggregation of the gradients to eliminate misreports that are usually outliers [183]. However, the median based aggregation is computationally costly and faces convergence issues. As such, the study of [184] proposes DRACO, which is based on the coding of gradients and algorithmic redundancy, i.e., each worker evaluates redundant gradients, such that the accurate gradients may be derived even in the presence of adversarial nodes. The simulation results show that DRACO is more than 3 times faster in achieving 90% test accuracy for gradient computations on MNIST dataset as compared to the geometric median method.

An improvement to the studies of [20], [182], [184] is done in [21], which proposes LCC to achieve an optimum tradeoff between resiliency against straggling workers, security against malicious workers, and information-theoretic privacy amid colluding workers. In LCC, the dataset of the master

is encoded using the Lagrange polynomial to create computational redundancy. Then, the coded data is shared to the workers for computation on the encoded data, as if the coding did not take place. In comparison with the BGW MPC scheme [180], LCC requires more workers. However, the Lagrange polynomial based encoding leads to a reduction in the amount of randomness required to encode the data, which translates to lower storage and computation costs incurred by each worker. The LCC also outperforms the BGW based polynomial sharing [20] in terms of communication costs, given that the polynomial sharing scheme requires a communication round for each bilinear operations. In addition, LCC is less computationally costly than DRACO [184], which does not utilize the algebraic structure of the encoded gradients. However, the Lagrange coding only works for computations involving arbitrary multivariate polynomial functions of the input dataset. As an extension, the study of [186] proposes CodedPrivateML which adopts polynomial approximations to handle the non-linearities of the gradient computation when the sigmoid function is involved, such that logistic regression can be conducted on LCC-encoded data while providing information-theoretic privacy for the dataset. Given that the advantages of the LCC is preserved, the experiments conducted on Amazon EC2 clusters validate that the proposed scheme is close to 34 times faster than the BGW based MPC approaches.

In light of the growing popularity of machine learning, the study of [185] proposes Harmonic coding for tasks specific to gradient-type computations, e.g., for loss function minimization in distributed model training. Harmonic coding leverages on the structure of the intermediate partial gradients computed to enable the cancelation of redundant results, such that the encoding and decoding process is more efficient. As such, for the same level of privacy constraint, Harmonic coding improves on Shamir's secret sharing scheme [187] and LCC [21] in terms of requiring fewer workers to compute gradient-type functions. This result is further summarized in Table VIII, where we present a comparison of the minimum number of workers required for the discussed schemes. Note that K refers to the number of partitions of the input dataset, g refers to the fixed multivariate polynomial, and $\deg$ refers to the degree of g . Like LCC, Harmonic coding can also be applied universally to any gradient-type function computation. As such, the data encoding can be performed before the computing task is specified, thus further reducing the delay in computation.

B. Secure Distributed Matrix Multiplication (SDMM)

Matrix multiplication is a key operation in many popular machine learning algorithms [188], e.g., principal component analysis [95], support vector machines [189], and gradient-based computations. While the reviewed studies in Section VI-A discuss coded computing for privacy preservation in general operations, the studies to be discussed consider tailored strategies for SDMM.

In [190] and [188], the authors propose the use of staircase codes in place of linear secret sharing codes, e.g., Shamir's

codes [187]. As an illustration, we consider that a master encodes its data A with random matrix R into three secret shares before transmitting a share to each of the workers to perform matrix multiplication. When linear secret sharing codes are used, the data and random matrix are not segmented but instead encoded and transmitted as a whole (Table IX). As such, the master has to wait for two *full* responses from any two of three workers before being able to decode and derive the desired results. In contrast, when the staircase code is used, the data and random matrices are segmented into sub-shares before transmission to the workers. When sufficient sub-tasks have been completed by the workers, the master can then instruct the workers to cease computation. Clearly, the staircase coding approach reduces the computation cost of workers and communication costs incurred by the master. Accordingly, the staircase coding approach can outperform the classical secret sharing code in terms of mean waiting time. With 4 workers considered, experiments conducted on the Amazon EC2 clusters show a 59% decrease in mean waiting time using staircase codes.

However, [188] and [190] still consider the case of one-sided privacy, i.e., the approach is designed to keep only one of two input matrices involved in SDMM operations private. As such, several studies have shifted the focus towards applying coding techniques to the specific case in which two-sided privacy, i.e., in which both input matrices are private, is ensured. Among the first such study is that of [179], which applies the aligned secret sharing for two-sided privacy. Specifically, the input matrices are split into submatrices and encoded with random matrices. Then, the undesired terms are aligned such that the server only recovers the desired results and saves on communication costs. This leads to an improved download rate, i.e., the ratio of size of desired result to the amount of information downloaded by a user, over conventional secret sharing schemes.

Following [179], the study of [191] proposes an inductive approach to find a close-to-optimal partition of the input matrices in consideration of two metrics namely the download rate and the minimum number of required workers. The proposed scheme improves on download rate, number of tolerable colluding servers, and computational complexity as compared to the study of [179]. Inspired by [83], the polynomial coding scheme is also extended to SDMM operations and specifically convolution tasks in [192], while preserving two-sided privacy, download rate similar to that of [191], and further mitigating the straggler effects. For convolution tasks, the authors leverage on the inherent property in which the sums of convolutions of sub-vectors may be used to derive the convolution result. Then, the upper and lower bounds of the recovery threshold is derived to show that an order-optimal recovery threshold is achieved, i.e., it does not scale with number of workers.

However, the key weakness of [179] and [191] as indicated in [193] is that the proposed theoretical results do not clarify the effect of matrix dimensions on the download rate, i.e., the download rates are derived in the case whereby matrices are simply assumed to have large dimensions but without any specifications otherwise. However, the study of [193] found that the results for [179], [191] may be violated in some cases

TABLE IX
COMPARISON OF LINEAR SECRET SHARING CODES AND STAIRCASE
CODES IN THE DISTRIBUTION OF COMPUTATION TASKS IN A SYSTEM
WITH THREE WORKERS [188], [190]

	S_1	S_2	S_3
Linear secret sharing code	R	$R + A$	$R + 2A$
Staircase code	$A_1 + 2A_2 + 4R_1$ $R_1 + R_2$	$A_1 + 2A_2 + 4R_1$ $R_1 + 2R_2$	$A_1 + 2A_2 + 4R_1$ $R_1 + 3R_2$

for differing relative dimensions of the input matrices. Under this context, the model proposed in [193] allows the matrix dimensions to be specified, and a new converse bound for the two-sided security SDMM is derived.

In general, the encoded results of matrix multiplication are sent to the master, where interpolation is performed to obtain the multiplication results, i.e., coefficients of a polynomial. In [179], [191], the encoding of the private matrices is such that the coefficients are mainly non-zero. In contrast, the study of [194] propose the Gap Additive Secure Polynomial (GASP) codes such that there are as many zero coefficients as possible. This allows the product to be interpolated and the desired results to be derived with fewer number of evaluations performed, which implies that fewer workers are required to perform the matrix multiplication. To assign the exponents for decodability while having as many zero coefficients as possible, the authors propose the degree table to solve the combinatorial problem. In [195], the authors further generalize the GASP codes to be applicable to different partitions of input matrices and security parameters. The GASP codes are shown to outperform the approaches in [179], [191] in terms of download rate.

C. Summary and Lessons Learned

In this section, we have discussed studies that have adopted a coded computing approach towards ensuring secure distributed computing. Then, we discuss the specific applications of SDMM, which require two-sided privacy. The summary and lessons learned are as follows.

- Originally proposed to mitigate stragglers in distributed computing systems, coding approaches have also been extended to ensure information theoretic privacy [20] and for some studies, security against malicious workers [21], [184]. However, given that the coding techniques, e.g., polynomial codes, are designed with the intention to mitigate stragglers, the studies discussed in this section have focused on modifications to the approaches to simultaneously achieve both straggler mitigation and privacy. As was mentioned in this section, the coding approaches to ensure privacy preservation have outperformed conventional secret sharing schemes that mainly focused on data correctness and privacy.
- With the recent popularity of machine learning, several studies have also focused on tailoring the coding approaches to be utilized for various machine learning tasks, e.g., entangled polynomial coding for logistic regression [182] and harmonic coding for gradient based computations [185]. In particular, given the importance of

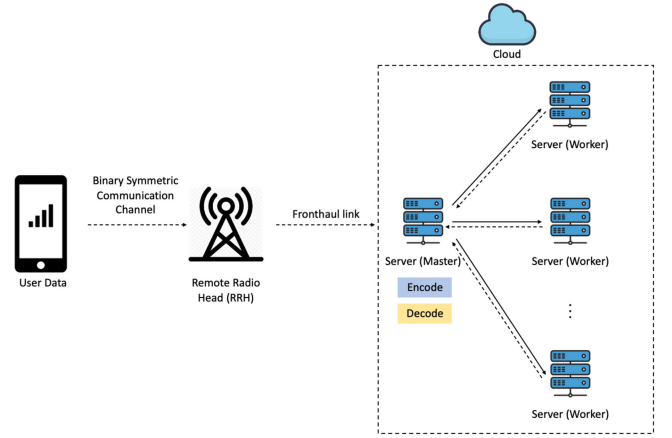


Fig. 11. Application of CDC to Network Function Virtualization (NFV) model for uplink channel decoding.

two-sided privacy in SDMM, we have also specifically discussed papers that focus on two-sided privacy separate from papers that only considered one-sided privacy. In general, papers that focus on taking advantage of the algebraic structure of the specific operations [21], e.g., convolution tasks [192] or gradient computations [185], for efficient encoding and decoding usually perform better.

- In most of the studies that we have discussed, the focus tends to be on information-theoretic privacy. However, weak security is key and may be explored in future works. Specifically, a system is weakly secure if attackers are unable to learn the sensitive intermediate values without having received a certain number of coded packets [196]. This may be relevant when there exist eavesdroppers with an access to communication links between the master and workers [192]. For example, in the study of [197], a data shuffling design and redundancy reduction algorithm to assign computing tasks have been proposed to ensure weak security in the system. However, the focus has been on workers, rather than eavesdroppers that may not be involved in the computations.
- Given that the straggler effect is a fundamental concern in distributed computing, most papers have considered workers with heterogeneous computational capabilities. However, as discussed in [192], the issue of heterogeneous networks in other aspects have been under-explored in the aforementioned studies. For example, the workers may have different levels of reputation [198]. This enables the adoption of context-sensitive solutions, e.g., data security may be guarded against new workers or workers with low reputation, whereas it may not be required for trusted workers. With this, the computation complexity and duration may be reduced in trusted nodes.

VII. CDC APPLICATIONS

A. Network Function Virtualization (NFV)

Network Function Virtualization serves as an enabling technology for optimizing the 5G as well as emerging 6G

networks [199], presenting a promising paradigm shift in the telecommunication service provisioning industry. By leveraging on virtualization technologies, NFV simplifies the management and operations of the networking services. In particular, NFV decouples the Network Functions (NFs) such as routing and baseband processing from the physical network equipment on which they operate. The NFs are mapped to the Virtual Network Functions (VNFs) that are supported by the Commercial Off-The-Shelf (COTS) physical resources which provide storage, networking and computational capabilities. As the software component in the network is decoupled from the hardware component by a virtualization layer, the VNFs can be easily implemented over the distributed network locations which have the required hardware resources. Due to the flexibility in the deployment of the VNFs, the NFV brings about significant reduction in operating and capital expenses. Moreover, the development of new networking services is faster and cheaper as the COTS resources can be instantiated easily to provide the required network connection services. Extensive literature has been carried out on the various aspects of NFV such as architectural designs [199], [200], resource allocation [201], [202], energy efficiency [203], performance improvement [204] and security [205], [206]. More details can be found in [199], [207], [208].

However, one of the limiting factors of the performance of NFV lies in the reliability of the COTS hardware resources [199], [209]. Hardware failure due to several factors such as component malfunctioning, temporary unavailability and malicious attacks affects the implementation of NFV, hindering the provision of services. Apart from the fault-tolerant virtualization strategies that are based on the diversity approach which maps VNFs onto the various virtual machines (VMs) such that the probability of a disruptive failure is minimized [207], coding approach can also be used to minimize computation latency in NFV.

In the study of [210], the authors consider the highly complex uplink channel decoding of the Cloud Radio Access Network (C-RAN) architecture [211], which is a key application of NFV. In this system, the users communicate with the cloud via Remote Radio Head (RRH). In order to ensure the reliability of channel decoding, the received data frames by the RRH are encoded by leveraging on their algebraic structures before being distributed to the different VMs as shown in Fig. 11. The simulation results show that the coding approach achieves lower probability of error for decoding at the cloud than that of the diversity-based approach. However, there are several assumptions that are made in this proposed scheme. Firstly, the binary symmetric communication channel is assumed between the users and the RRH. By considering other communication channels such as the additive Gaussian noise channels, different coding techniques may be applied. Secondly, this simple framework works well for a network with three computing nodes, but its performance for larger-size networks is not guaranteed.

Considering the same issue of uplink channel decoding in the C-RAN architecture, the authors in [212] propose a more generalized coded computation framework that works for any number of servers, random computing runtimes and random

packet arrivals by adopting the coding approach proposed in [210]. Given the randomness in the arrivals of data frames transmitted by the users, two queue management policies are considered: (i) per-frame decoding where one frame is decoded at any point of time, and (ii) continuous decoding where the servers start to decode the next packet of data frame upon completion of the first packet. There is an underlying tradeoff between the average decoding latency and the frame unavailability probability which is an indication of the reliability of the decoding process at the servers. The simulation results show that properly designed NFV codes are useful in achieving the desired tradeoffs by optimizing the minimum distance of the codes.

The idea of adopting coding approach in NFV is relatively new. Different coding techniques can be explored in the future. Besides, instead of addressing the uplink channel decoding, which is the most computationally-intensive baseband function [211], other network functions such as routing and security functions can be considered.

B. Edge Computing

With the enhanced sensing capabilities of end devices, an overwhelming amount of data is produced at the edge of the network today. Traditional schemes of computation offloading to the cloud is thus unsustainable. Moreover, certain edge applications may involve end devices in remote areas that have limited connectivity. This necessitates a paradigm shift towards edge computing, in which computation is performed closer to the edge of the network where data is produced. However, resource-constrained devices may not be able to carry out complex computations individually, especially given the increasing size and complexity of state-of-the-art AI models [213]. As such, one of the enabling technologies of edge computing is cooperative computation in which the available resources of end devices and edge nodes, e.g., road side units in vehicular networking, can be pooled together to execute computation intensive tasks collaboratively [26]. For ease of exposition, we refer to these participating end devices and edge nodes as workers in this subsection.

As the number of devices connected to the network increases, more information needs to be exchanged among the workers, resulting in high communication load. However, the communication bandwidth is fixed and thus the network is unable to handle the high communication load, causing a bottleneck as a result. Moreover, the heterogeneous nature of workers in the edge computing paradigm, e.g., in terms of computational and communication capabilities, can lead to the straggler effects. In face of these challenges, coding techniques can be used.

In the study of [214], the *coded wireless distributed computing* (CWDC) framework is proposed. The system model consists of multiple devices, i.e., workers, involved in cooperative computation. As an illustration, a worker may have an input, e.g., an image, that has to be processed, e.g., for object recognition. Individually, a worker may not have the storage or computational capabilities to execute the task. Therefore, the inference model may be split and stored on each worker,

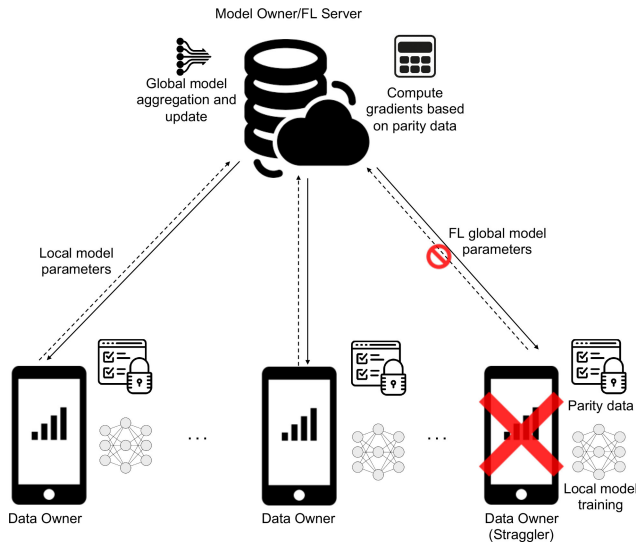


Fig. 12. Illustration of the Coded Federated Learning (CFL) scheme in an FL system that consists of an FL model owner and multiple data owners.

whereas the cooperative computation of results can be implemented following the MapReduce framework as discussed in Section II-A. An access point, e.g., a base station or a Wi-Fi router can then be utilized to facilitate the exchange of intermediate results among workers. The proposed framework achieves communication loads that are independent of the size of the network and the storage size of the workers. Moreover, the CWDC framework can be generalized and applied to different types of applications.

In practical distributed computing systems, the workers may have heterogeneous computational, communication, and storage capabilities. Based on a similar system model proposed in [214] where the workers communicate with each other via an access point, the study in [215] considers devices with heterogeneous storage capacities over wireless networks. For uplink transmission, the allocation of files is based on the scheme proposed in [112] as previously discussed in Section IV-A whereas for downlink transmission, data is encoded at the access point for the reduction of downlink communication load. However, the achievable scheme has only been validated in a small network that consists just three computing nodes. In the work of [216], the authors consider a master-aided edge computing network by incorporating the master in the completing the computation tasks. This proposed scheme reduces communication time and achieves an optimal tradeoff between the overall communication time, computation load and master's storage capacity.

In light of the growing popularity of machine learning model training at the edge, the study of [217] considers the distribution of gradient descent computations across workers in the network to train a linear regression model. The proposed heterogeneous coded gradient descent (HCGD) assigns each worker with an optimal load partition, through modeling the computation delay of devices with a shifted exponential distribution. In consideration of data privacy, the authors in [218] propose the Coded Federated Learning (CFL) approach for privacy-preserving linear regression. Federated Learning (FL)

is a privacy preserving distributed machine learning paradigm proposed in [219], in which the sensitive data of data owners are kept locally, whereas only the model parameters are transmitted to the FL server for aggregation to update the model. In particular, the FL global model parameters are sent from the FL server to the data owners. Then, the data owners train the FL model on their own datasets. Upon completion, the data owners transmit the local model parameters to the FL server for aggregation. The process iterates for a number of times until a convergence is achieved. However, FL still suffers from the issues of straggling devices and communication inefficiency [173]. As such, in the proposed CFL scheme, each data owner first generates parity data from its local data for transmission to the FL server. At the FL server, the gradients are also computed from parity data simultaneously, such that only a subset of gradients from the data owners have to be received for completion of model update. Figure 12 illustrates the implementation of the CFL scheme. The CFL approach is also shown to converge almost four times faster than an uncoded approach. However, the communication and computation cost involved in generating and transmitting the parity data has not been well elaborated in the study. Clearly, a major weakness of aforementioned studies of [217], [218] is that they can only be applied to linear regression model training. The study of [220] extends on the aforementioned works with the proposed CodedFedL designed to mitigate straggler mitigation in non-linear regression and classification tasks.

In some cases, the resource level of the devices may not be known by the network operator. To enable a dynamic and adaptive coded sub-task allocation for cooperative computation, an Automatic Repeat reQuest (ARQ) mechanism is proposed in the study of [149], in which devices are allocated with specific levels of packets for computation based on their responsiveness. Specifically, devices that are more responsive are assumed to have more available resources. These devices will thus be assigned with more sub-tasks for computation.

Instead of reducing communication load, it is also important to consider communication efficiency, i.e., achieved data rates, especially in wireless networks that have limited spectral resources or networks with mutual interference among users. In order to improve spectral efficiency, the co-channel communication model is proposed in the study of [221] which consists of two stages, i.e., the uplink multiple access stage and the downlink broadcasting stage. The communication model turns out to be equivalent to a multiple-input-multiple-output (MIMO) interference channel. Interference alignment [222] has been an effective approach in handling mutual interference among users. The signals are precoded into the same subspaces at the unintended receivers and the desired signals are recovered at the intended receivers by using the decoding matrix. Linear coding scheme is adopted to establish the conditions for interference alignment. A low-rank optimization problem is formulated to minimize the number of channel used subject to the established interference alignment conditions. By solving this optimization problem, the achievable symmetric degree-of-freedom (DoF), which implies the extent to which interference is eliminated, can be maximized. In [221], an efficient difference-of-convex-functions (DC) algorithm based on

a DC representation for the rank function is proposed to solve the low-rank optimization problem. The performance of the DC algorithm is evaluated in two different scenarios by varying: (i) the number of files stored in the devices, and (ii) the number of antennas that are equipped by the devices. In the first scenario, as the number of files stored in the devices increases, the achievable DoF increases. Similarly in the second scenario, as the number of antennas equipped by the devices increases, the achievable DoF increases. Furthermore, the simulation results show that the DC approach achieves higher DoF than the existing benchmark algorithms, e.g., iterative reweighted least squares (IRLS) algorithm and the nuclear norm relaxation approach. However, the proposed scheme is based on a homogeneous network, i.e., the number of files stored is the same across all devices and all devices are equipped with the same number of antennas. As an extension, heterogeneous networks can be considered.

Instead of communicating with each other via an access point, the devices can communicate directly with each other over wireless interference channels [223]. In particular, the transmission of data in the Shuffle phase operates over wireless interference channels. While the CDC scheme in [17] allows communications based on time-division multiple access (TDMA) scheme which allows each computing node to transmit one coded information packet at any time, the one-shot linear scheme adopted in [223] allows more than one computing node to transmit information simultaneously at any given time slot. The transmitted symbols are linear combinations of coded intermediate results from the computing nodes. The transmitted symbols are broadcasted to the computing nodes, following which the nodes can decode to recover the desired information. The study in [223] characterizes an improved computation-communication tradeoff as compared to the study in [17]. However, the proposed scheme operates under the assumption of perfect channel state information (CSI) where the CSI is available to all computing nodes. As such, the authors in [224] propose superposition coding scheme which has better performance than that of the CDC scheme [17] and the one-shot linear scheme [223] under imperfect CSI condition. The study is extended to account for the presence of stragglers in the networks [225].

In general, the aforementioned studies have considered conventional implementations of the CDC scheme in which each device has to complete the computation task first, before transmitting the intermediate results, e.g., to other devices via the access point. However, given the limited computational capabilities of devices, the latency involved can still be significant. As such, the studies in [152] and [226] consider the batch-processing based coded computing (BPCC) framework which allows each device to return only the partially completed results to a master in batches, even before the task is fully completed. These partial results are then utilized to generate approximations to the full solution through the singular value decomposition (SVD) approach [151]. This is particularly useful in applications that require fast, but not necessarily optimized, results. The BPCC scheme has its effectiveness validated on the EC2 computing cluster, in which latency is proven to be reduced. Moreover, in consideration of the energy

limitations of Unmanned Aerial Vehicles (UAV), the BPCC framework was proposed for UAV-based mobile edge computing to provide energy-efficient computation offloading support on-demand [227], [228].

C. Summary and Lessons Learned

In this section, we have discussed applications of CDC in NFV and edge computing. The lessons learned are as follows.

- The convergence of the recently popular edge computing and machine learning has given rise to edge intelligence, in which the computational capabilities of edge servers and devices have been increasingly leveraged to conduct machine learning model training closer to where the data is produced. While this leads to several benefits, e.g., lower training latency and enhanced privacy preservation, the problem of straggling devices is still a bottleneck. As such, CDC approaches have been increasingly applied in this context.
- One major difference between distributed computing at the edge and at computing clusters is that the end devices and edge servers, i.e., workers, are not specifically dedicated to computations. For example, the workers may only share a fraction of their processing power [217] at time slots in which they are idle. As such, the heterogeneity in computational capabilities of workers may also be greater in this regard. In face of these challenges, optimal load partition and allocation strategies have been explored. Moreover, for the future works, the dynamics of the system may be captured using Deep Reinforcement Learning based resource allocation [229].
- FL has been applied to develop the future vehicular IoT systems such as intelligent transportation systems and collaborative autonomous driving systems in order to train the FL model while preserving data privacy [230]. In the FL network, there may be several data owners involved. Moreover, the data owners can be vehicles with computational constraints. As such, CFL [218] and CodedFedL [220] have been proposed to mitigate the straggler effects in FL. However, the proposed methods are highly restrictive. For example, the CFL can only be applied to linear regression problems, whereas both methods require costly computation and transmission of parity data. Moreover, they have not been implemented on typical end devices to validate the feasibility of the schemes in practical implementation. For the future works, studies on using CDC schemes in edge computing applications may adopt the approach of [152], [226], in which the schemes are implemented under the context of practical hardware constraints.

VIII. CHALLENGES, OPEN ISSUES AND FUTURE WORKS

The utilization of CDC schemes to solve the implementation challenges of the distributed computing systems is a relatively new and recent approach. There are still challenges and open issues that have yet to be addressed and this provides opportunities for new research directions in the future.

We present major challenges that need to be looked into for effective implementation of CDC schemes.

- *Heterogeneous nodes:* As compared to traditional distributed computing clusters, heterogeneities among computing nodes are much more significant when the computing nodes are connected in the edge computing networks, e.g., smartphones and wearable devices. Many studies, e.g., [112], [134], [136], [146] have considered heterogeneous systems where the computing nodes have different computational, communication and storage capabilities. For example, in [136], a joint file and function allocation strategy is proposed to assign jobs to the computing nodes such that the communication load is minimized. Similarly, the authors in [231] have proposed a low-complexity combinatorial design for cascaded CDC, of which the performance is better in heterogeneous networks than in its homogeneous counterpart. In [146], the computation load is allocated based on the capabilities of the computing nodes. However, other aspects of heterogeneity such as the reputation [198] and the willingness to participate of the computing nodes are not taken into account. New allocation strategies need to consider different aspects of heterogeneities of the processing devices so that coding techniques can be implemented effectively and securely. For example, the computation tasks can be allocated to workers with higher reputation which implies higher probability of the workers in completing their allocated tasks.
- *Encoding and decoding complexities:* The studies that we have discussed in Section IV and Section V mainly minimize communication load in Shuffle phase and computation latency in the Map phase respectively. However, the complexities of encoding and decoding are often not evaluated. It is important to ensure low encoding and decoding complexities in order to minimize the overall job execution time. Otherwise, the speedup gain achieved by in specific phases, e.g., communication and computation phases may be offset by the high encoding and decoding complexities. For example, the UberShuffle algorithm [133] incurs high computational overhead under the fast broadcast environment, i.e., networks with large bandwidth, such that it is not feasible for implementation even though it achieves significant shuffling gain. Hence, to better assess the performance of the CDC schemes, the complexities of the encoding and decoding methods have to be evaluated. One proposed solution is the SAZD-based CDC scheme [232] which reduces the number of multiplication and division operations in the encoding and decoding process. In particular, shift-and-addition (SA) is proposed to implement the linear independence in the encoding process. In the SA algorithm, the elements are shifted right by several elements and then added up over the real field in an elementwise manner. On the other hand, zigzag decoding (ZD) is used in the decoding process where backward substitution and addition operations replace the expensive multiplications and divisions.
- *Non-static computing nodes:* For the commonly used distributed computing models such as cluster computing, grid computing and cloud computing that are discussed in Section II, the computing nodes are static, i.e., the nodes are located at fixed location. For example, the servers are located at specific data centers. Data required for computations is transmitted over the wireless communication channels to the servers. However, as the edge devices, e.g., IoT devices, wearable devices and vehicles, have greater communication and computational capabilities, new distributed computing models such as mobile edge computing [27], [233] and fog computing [234] have been developed recently. In [235], the basic CDC scheme is implemented in the context of fog computing. However, the edge devices are usually mobile. The data that is processed by the edge devices depends on the locations that they visit [135], and hence the master has no control of the data distribution to the workers. New coding approaches for edge and fog computing which involve moving workers can be proposed in future works. One proposed solution is to allocate Reduce functions based on the data stored at the computing nodes [135].
- *Security concerns:* Coding techniques are able to mitigate the straggler effects while preserving privacy as shown in the studies of [20], [21], and [184]. The proposed secure coding techniques are an extension to the coding techniques that are originally proposed to mitigate the straggler effects. As mentioned in [21], the tradeoff between resiliency against straggling workers, security against malicious workers and information-theoretic privacy amid colluding workers needs to be carefully managed. In addition, there may exist eavesdroppers which tap on the less secure communication links between the master and the workers. As such, more research effort can be directed towards developing weakly secure systems which prevents the eavesdroppers from retrieving sensitive information.
- *Network architectures:* It is important to consider how the computing nodes are connected and communicate with each other for effective implementation of the CDC schemes. For example, in [114] and [236], the authors introduce a hierarchical structure where the master communicates with multiple submaster and each submaster leads a group of computing nodes. From the studies that we have reviewed, the network architecture is only considered for the implementation of CDC schemes to reduce communication load. However, it is an important consideration when designing secure coding schemes. In practice, it may be safe for computing nodes within a group, e.g., from the same location, to share information freely with each other but not with computing nodes from another group, e.g., from different location. In addition, the communication channels are not perfect, i.e., they may not have perfect CSI [224] or the transmitted information may have missing entries. Some networks may have limited spectral resources or suffer from mutual interference among users [221]. As such, future research can work towards designing coding schemes that can be

implemented in practical distributed computing systems. Besides a need to design effective coding schemes, there is also a need to consider the design of low-cost, easily-implementable and scalable network architectures in which the coding schemes can be applied to.

- *Incentive mechanisms:* In the CDC networks, the workers are usually self-interested and belong to different organizations. As such, effective game-theoretic incentive mechanisms are required to encourage the workers to participate in the distributed computation tasks. For example, in the work of [237], the master with a limited amount of budget aims to complete a coded distributed gradient descent computation problem with minimum latency. The interactions between the master and the workers are modeled as a Stackelberg game. In particular, the master and workers are the follower and leaders respectively. The leaders decide on the frequency of the CPU cycles whereas the follower determines the allocation of computation loads based on the decision of the leaders. The Stackelberg game is solved by backward induction.
- *Different computation frameworks:* Currently, most of the studies are based on the MapReduce computing model. Specifically, Coded MapReduce is proposed in [18] by implementing coding techniques in the MapReduce framework. However, there are limitations to MapReduce model that hinder its wide adoption for all types of distributed computation tasks as explained in Section II-A. In fact, there are other computing models such as Spark, Dryad and CIEL which support iterative algorithms, in which the feasibility of implementing coding techniques has not been explored. As such, the importance of these computing models motivates future directions such as the design of coding schemes that are specific to these computing models in order to solve any distributed computation tasks, e.g., convolution, Fourier transform and non-linear computations.
- *Coding for both communication reduction and stragglers mitigation:* As we have discussed previously in Section IV and Section V, coding techniques are used to either reduce communication load or mitigate the straggler effects. However, coding techniques cannot be applied to solve these implementation challenges simultaneously. As characterized in [104], there is a tradeoff between communication load and computation latency. However, in an ideal situation, both communication load and computation latency need to be minimized. Thus, it is important to carefully manage the tradeoff to achieve optimal performance of the distributed computing systems. For future works on CDC schemes, there is a need to improve the latency-communication tradeoff curve so that the time taken to execute the allocated tasks can be significantly reduced. In addition, instead of the two-dimensional tradeoff, the three-dimensional tradeoff between computation, communication and storage cost [74], which is much more challenging to manage, should be considered.
- *CDC applications:* Given the advantages, CDC schemes are implemented in distributed computing applications

such as the NFV and edge computing. Apart from the UAVs, CDC schemes can be extended to edge computing applications in other areas, e.g., vehicular networks, healthcare systems and industrial operations. For the studies that we have reviewed in CDC applications, the main focus lies in the implementation of coding techniques in various applications, without considering privacy and security. Given the importance of secure coding as discussed in Section VI, secure coding techniques need to be considered in the implementation of CDC applications. Besides, application-specific issues need to be addressed. For example, in vehicular networks where the vehicles are constantly moving, the CDC schemes need to be robust to vehicles which do not have consistent access to the wireless communication channels. In addition, there are multiple masters with different computation tasks to complete at the same time in practical applications. As such, a multi-task assignment problem [238] in the CDC networks should be considered to solve the computation load allocation problem.

The idea of using coding techniques to overcome the challenges in distributed computing systems is relatively new. For effective implementation in practical distributed computing systems, various aspects such as the heterogeneities of the computing nodes and the network architectures are worth the in-depth studies. Some promising research directions presented in this survey serve as useful guidelines and valuable references for future research in CDC.

IX. CONCLUSION

In this paper, we provided a tutorial of CDC schemes and a comprehensive survey on the two main lines of CDC works. We first motivated the need for CDC schemes. The current performance of distributed computing systems can be improved using coding schemes. Then, we described the fundamentals and principles of CDC schemes. We also reviewed CDC works which aim to minimize communication costs, mitigate straggler effects as well as enhance privacy and security. In addition, we discussed the implementation of CDC schemes in practical distributed computing applications. Finally, we highlighted the challenges and discussed promising research directions.

REFERENCES

- [1] V. Cristea, C. Dobre, C. Stratan, F. Pop, and A. Costan, *Large-Scale Distributed Computing and Applications: Models and Trends*. London, U.K.: IGI Global, 2010.
- [2] V. P. Kumar, V. K. Prasanna, S. Iyengar, P. Spirakis, and M. Welsh, *Distributed Computing in Sensor Systems*. New York, NY, USA: Springer, 2005.
- [3] J. C. Corbett et al., "Spanner: Google's globally distributed database," *ACM Trans. Comput. Syst.*, vol. 31, no. 3, pp. 1–22, 2013.
- [4] J. Gonzaga, L. Meleiro, C. Kiang, and R. M. Filho, "ANN-based soft-sensor for real-time process monitoring and control of an industrial polymerization process," *Comput. Chem. Eng.*, vol. 33, no. 1, pp. 43–49, 2009.
- [5] D. Giusti and A. Eduardo, "Structured parallel programming: Patterns for efficient computation," *J. Comput. Sci. Technol.*, vol. 15, no. 1, pp. 43–44, 2015.
- [6] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Commun. ACM*, vol. 51, no. 1, pp. 107–113, 2008.

- [7] F. Ahmad, S. T. Chakradhar, A. Raghunathan, and T. N. Vijaykumar, "Tarazu: Optimizing MapReduce on heterogeneous clusters," *ACM Special Interest Group Comput. Architect. (SIGARCH) Comput. Architect. News*, vol. 40, no. 1, pp. 61–74, 2012.
- [8] Y. Guo, J. Rao, D. Cheng, and X. Zhou, "iShuffle: Improving hadoop performance with shuffle-on-write," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 6, pp. 1649–1662, Jun. 2017.
- [9] M. Chowdhury, M. Zaharia, J. Ma, M. I. Jordan, and I. Stoica, "Managing data transfers in computer clusters with orchestra," *ACM Special Interest Group Data Commun. (SIGCOMM) Comput. Commun. Rev.*, vol. 41, no. 4, pp. 98–109, 2011.
- [10] Z. Zhang, L. Cherkasova, and B. T. Loo, "Performance modeling of MapReduce jobs in heterogeneous cloud environments," in *Proc. IEEE 6th Int. Conf. Cloud Comput.*, Santa Clara, CA, USA, 2013, pp. 839–846.
- [11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [13] M. A. Attia and R. Tandon, "Combating computational heterogeneity in large-scale distributed computing via work exchange," 2017. [Online]. Available: arXiv:1711.08452.
- [14] D. Wang, G. Joshi, and G. Wornell, "Using straggler replication to reduce latency in large-scale parallel computing," *Special Interest Group Meas. Eval. (SIGMETRICS) Perform. Eval. Rev.*, vol. 43, no. 3, pp. 7–11, 2015.
- [15] R. K. Maity, A. S. Rawa, and A. Mazumdar, "Robust gradient descent via moment encoding and LDPC codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Paris, France, 2019, pp. 2734–2738.
- [16] M. A. Maddah-Ali and U. Niesen, "Fundamental limits of caching," *IEEE Trans. Inf. Theory*, vol. 60, no. 5, pp. 2856–2867, May 2014.
- [17] S. Li, M. A. Maddah-Ali, Q. Yu, and A. S. Avestimehr, "A fundamental tradeoff between computation and communication in distributed computing," *IEEE Trans. Inf. Theory*, vol. 64, no. 1, pp. 109–128, Jan. 2018.
- [18] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "Coded MapReduce," in *Proc. 53rd Annu. Allerton Conf. Commun. Control Comput. (Allerton)*, Monticello, IL, USA, 2015, pp. 964–971.
- [19] K. Lee, M. Lam, R. Pedarsani, D. Papailiopoulos, and K. Ramchandran, "Speeding up distributed machine learning using codes," *IEEE Trans. Inf. Theory*, vol. 64, no. 3, pp. 1514–1529, Mar. 2018.
- [20] H. A. Nodehi and M. A. Maddah-Ali, "Limited-sharing multi-party computation for massive matrix operations," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Vail, CO, USA, 2018, pp. 1231–1235.
- [21] Q. Yu, S. Li, N. Raviv, S. M. M. Kalan, M. Soltanolkotabi, and S. A. Avestimehr, "Lagrange coded computing: Optimal design for resiliency, security, and privacy," in *Proc. 22nd Int. Conf. Artif. Intell. Stat. (AISTATS)*, Naha, Japan, 2019, pp. 1215–1225.
- [22] F. Kalim, J. P. Jeong, and M. U. Ilyas, "CRATER: A crowd sensing application to estimate road conditions," *IEEE Access*, vol. 4, pp. 8317–8326, 2016.
- [23] S. Jovanović et al., "A mobile crowd sensing application for hypertensive patients," *Sensors*, vol. 19, no. 2, p. 400, 2019.
- [24] J. Dutta, C. Chowdhury, S. Roy, A. I. Middy, and F. Gazi, "Towards smart city: Sensing air quality in city based on opportunistic crowd-sensing," in *Proc. 18th Int. Conf. Distrib. Comput. Netw. (ICDCN)*, 2017, p. 42.
- [25] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [26] W. Y. B. Lim et al., "Incentive mechanism design for resource sharing in collaborative edge learning," 2020. [Online]. Available: arXiv:2006.00511.
- [27] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 4th Quart., 2017.
- [28] K. Krauter, R. Buyya, and M. Maheswaran, "A taxonomy and survey of grid resource management systems for distributed computing," *Softw. Pract. Exp.*, vol. 32, no. 2, pp. 135–164, 2002.
- [29] H. Hussain et al., "A survey on resource allocation in high performance distributed computing systems," *Parallel Comput.*, vol. 39, no. 11, pp. 709–736, 2013.
- [30] D. Datla et al., "Wireless distributed computing: A survey of research challenges," *IEEE Commun. Mag.*, vol. 50, no. 1, pp. 144–152, Jan. 2012.
- [31] S. P. Ahuja and J. R. Myers, "A survey on wireless grid computing," *J. Supercomput.*, vol. 37, no. 1, pp. 3–21, 2006.
- [32] G. L. Valentini et al., "An overview of energy efficiency techniques in cluster computing systems," *Clust. Comput.*, vol. 16, no. 1, pp. 3–15, 2013.
- [33] N. Sadashiv and S. M. D. Kumar, "Cluster, grid and cloud computing: A detailed comparison," in *Proc. 6th Int. Conf. Comput. Sci. Educ. (ICCSE)*, Singapore, 2011, pp. 477–482.
- [34] H. Kamal Idrissi, A. Kartit, and M. El Marraki, "A taxonomy and survey of cloud computing," in *Proc. Nat. Security Days (JNS3)*, 2013, pp. 1–5.
- [35] Y. Xu and H. Qi, "Distributed computing paradigms for collaborative signal and information processing in sensor networks," *J. Parallel Distrib. Comput.*, vol. 64, no. 8, pp. 945–959, 2004.
- [36] S. U. Khan, A. Y. Zomaya, and A. Abbas, *Handbook of Large-Scale Distributed Computing in Smart Healthcare*. Cham, Switzerland: Springer, 2017.
- [37] B. Tang et al., "Incorporating intelligence in fog computing for big data analysis in smart cities," *IEEE Trans. Ind. Informat.*, vol. 13, no. 5, pp. 2140–2150, Oct. 2017.
- [38] N. R. S. Raghavan and T. Waghmare, "DPAC: An object-oriented distributed and parallel computing framework for manufacturing applications," *IEEE Trans. Robot. Autom.*, vol. 18, no. 4, pp. 431–443, Aug. 2002.
- [39] A. A. Juan, J. Faulin, J. Jorba, J. Caceres, and J. M. Marquès, "Using parallel & distributed computing for real-time solving of vehicle routing problems with stochastic demands," *Ann. Oper. Res.*, vol. 207, no. 1, pp. 43–65, 2013.
- [40] I. Ahmad, M. K. Dhodhi, and A. Ghafoor, "Task assignment in distributed computing systems," in *Proc. Int. Phoenix Conf. Comput. Commun.*, 1995, pp. 49–53.
- [41] M. Kafil and I. Ahmad, "Optimal task assignment in heterogeneous distributed computing systems," *IEEE Concurrency*, vol. 6, no. 3, pp. 42–50, Jul.–Sep. 1998.
- [42] S.-H. Woo, S.-B. Yang, S.-D. Kim, and T.-D. Han, "Task scheduling in distributed computing systems with a genetic algorithm," in *Proc. High Perform. Comput. Inf. Superhighway (HPC Asia)*, Seoul, South Korea, 1997, pp. 301–305.
- [43] R. V. Lopes and D. Menascé, "A taxonomy of job scheduling on distributed computing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 12, pp. 3412–3428, Dec. 2016.
- [44] R. Ranjan, A. Harwood, and R. Buyya, "A case for cooperative and incentive-based federation of distributed clusters," *Future Gener. Comput. Syst.*, vol. 24, no. 4, pp. 280–295, 2008.
- [45] L. Duan, T. Kubo, K. Sugiyama, J. Huang, T. Hasegawa, and J. Walrand, "Incentive mechanisms for smartphone collaboration in data acquisition and distributed computing," in *Proc. IEEE INFOCOM*, Orlando, FL, USA, 2012, pp. 1701–1709.
- [46] Y. Xiao, *Security in Distributed, Grid, Mobile, and Pervasive Computing*. Hoboken, NJ, USA: CRC Press, 2007.
- [47] S. Pillana, I. Brandic, and S. Benkner, "Performance modeling and prediction of parallel and distributed computing systems: A survey of the state of the art," in *Proc. 1st Int. Conf. Complex Intell. Softw. Intensive Syst. (CISIS)*, Vienna, Austria, 2007, pp. 279–284.
- [48] D. Jiang, B. C. Ooi, L. Shi, and S. Wu, "The performance of MapReduce: An in-depth study," *Proc. VLDB Endow.*, vol. 3, nos. 1–2, pp. 472–483, 2010.
- [49] M. Zaharia et al., "Spark: Cluster computing with working sets," in *Proc. HotCloud*, vol. 10, 2010, p. 95.
- [50] M. Isard, M. Budi, Y. Yu, A. Birrell, and D. Fetterly, "Dryad: Distributed data-parallel programs from sequential building blocks," in *Proc. 2nd ACM SIGOPS/EuroSys Eur. Conf. Comput. Syst. (EuroSys)*, 2007, pp. 59–72.
- [51] D. G. Murray, M. Schwarzkopf, C. Smowton, S. Smith, A. Madhavapeddy, and S. Hand, "CIEL: A universal execution engine for distributed data-flow computing," in *Proc. 8th ACM/USENIX Symp. Netw. Syst. Design Implement.*, Boston, MA, USA, 2011, pp. 113–126.
- [52] G. Joshi, E. Soljanin, and G. Wornell, "Efficient redundancy techniques for latency reduction in cloud systems," *ACM Trans. Model. Perform. Eval. Comput. Syst.*, vol. 2, no. 2, pp. 1–30, 2017.
- [53] J. Weets, M. K. Kakhani, and A. Kumar, "Limitations and challenges of HDFS and MapReduce," in *Proc. Int. Conf. Green Comput. Internet Things (ICGCIoT)*, Noida, India, 2015, pp. 545–549.

- [54] *Hadoop Terasort*. Accessed: Aug. 13, 2020. [Online]. Available: <https://hadoop.apache.org/docs/current/api/org/apache/hadoop/examples/terasort/package-summary.html>
- [55] B. Recht and C. Ré, "Parallel stochastic gradient algorithms for large-scale matrix completion," *Math. Program. Comput.*, vol. 5, no. 2, pp. 201–226, 2013.
- [56] L. Bottou, "Stochastic gradient descent tricks," in *Neural Networks: Tricks of the Trade*. Heidelberg, Germany: Springer, 2012, pp. 421–436.
- [57] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," 2015. [Online]. Available: arXiv:1502.03167.
- [58] F. Ahmad, S. Lee, M. Thottethodi, and T. Vijaykumar, "MapReduce with communication overlap (MaRCO)," *J. Parallel Distrib. Comput.*, vol. 73, no. 5, pp. 608–620, 2013.
- [59] B. Nicolae, C. H. A. Costa, C. Misale, K. Katrinis, and Y. Park, "Leveraging adaptive I/O to optimize collective data shuffling patterns for big data analytics," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 6, pp. 1663–1674, Jun. 2017.
- [60] W. Yu, Y. Wang, X. Que, and C. Xu, "Virtual shuffling for efficient data movement in MapReduce," *IEEE Trans. Comput.*, vol. 64, no. 2, pp. 556–568, Feb. 2015.
- [61] M. Isard, V. Prabhakaran, J. Currey, U. Wieder, K. Talwar, and A. Goldberg, "Quincy: Fair scheduling for distributed computing clusters," in *Proc. ACM SIGOPS 22nd Symp. Oper. Syst. Principles (SOSP)*, 2009, pp. 261–276.
- [62] S. Suresh and N. Gopalan, "An optimal task selection scheme for hadoop scheduling," *IERI Procedia*, vol. 10, pp. 70–75, 2014.
- [63] J. Xie, F. Meng, H. Wang, H. Pan, J. Cheng, and X. Qin, "Research on scheduling scheme for Hadoop clusters," *Procedia Comput. Sci.*, vol. 18, pp. 2468–2471, 2013.
- [64] *Hadoop: Fair Scheduler*. Accessed: Jul. 6 2020. [Online]. Available: <https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/FairScheduler.html>
- [65] M. Zaharia, D. Borthakur, J. S. Sarma, K. Elmeleegy, S. Shenker, and I. Stoica, "Delay scheduling: A simple technique for achieving locality and fairness in cluster scheduling," in *Proc. 5th Eur. Conf. Comput. Syst. (EuroSys)*, Paris, France, 2010, pp. 265–278.
- [66] B. T. Rao and L. S. S. Reddy, "Survey on improved scheduling in Hadoop MapReduce in cloud environments," 2012. [Online]. Available: arXiv:1207.0780.
- [67] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013.
- [68] G. Ananthanarayanan et al., "Reining in the outliers in MapReduce clusters using Mantri," in *Proc. 9th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, Vancouver, BC, Canada, 2010, p. 24.
- [69] R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," *J. ACM*, vol. 46, no. 5, pp. 720–748, 1999.
- [70] K. Gardner, S. Zbarsky, S. Doroudi, M. Harchol-Balter, and E. Hyttia, "Reducing latency via redundant requests: Exact analysis," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 43, no. 1, pp. 347–360, 2015.
- [71] N. B. Shah, K. Lee, and K. Ramchandran, "When do redundant requests reduce latency?," *IEEE Trans. Commun.*, vol. 64, no. 2, pp. 715–722, Feb. 2016.
- [72] M. F. Aktas, P. Peng, and E. Soljanin, "Straggler mitigation by delayed ReLaunch of tasks," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 45, no. 3, pp. 224–231, 2018.
- [73] M. F. Aktas, P. Peng, and E. Soljanin, "Effective straggler mitigation: Which clones should attack and when?," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 45, no. 2, pp. 12–14, 2017.
- [74] Q. Yan, S. Yang, and M. Wigger, "Storage, computation, and communication: A fundamental tradeoff in distributed computing," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Guangzhou, China, 2018, pp. 1–5.
- [75] A. G. Dimakis, P. B. Godfrey, Y. Wu, M. J. Wainwright, and K. Ramchandran, "Network coding for distributed storage systems," *IEEE Trans. Inf. Theory*, vol. 56, no. 9, pp. 4539–4551, Sep. 2010.
- [76] N. Woolsey, X. Wang, R.-R. Chen, and M. Ji, "FLCD: A flexible low complexity design of coded distributed computing," 2020. [Online]. Available: arXiv:2004.03877.
- [77] S. Li, S. Supittayapornpong, M. A. Maddah-Ali, and P. Avestimehr, "Coded TeraSort," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, Lake Buena, FL, USA, 2017, pp. 389–398.
- [78] Y. H. Ezzeldin, M. Karmoose, and C. Fragouli, "Communication vs. Distributed computation: An alternative trade-off curve," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Kaohsiung, Taiwan, 2017, pp. 279–283.
- [79] A. Mallick, M. Chaudhari, U. Sheth, G. Palanikumar, and G. Joshi, "Rateless codes for near-perfect load balancing in distributed matrix-vector multiplication," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 3, no. 3, pp. 1–40, 2019.
- [80] S. Dutta, V. Cadambe, and P. Grover, "'Short-Dot': Computing large linear transforms distributedly using coded short dot products," *IEEE Trans. Inf. Theory*, vol. 65, no. 10, pp. 6171–6193, Oct. 2019.
- [81] S. Wang, J. Liu, N. Shroff, and P. Yang, "Fundamental limits of coded linear transform," 2018. [Online]. Available: arXiv:1804.09791.
- [82] K. Lee, C. Suh, and K. Ramchandran, "High-dimensional coded matrix multiplication," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Aachen, Germany, 2017, pp. 2418–2422.
- [83] Q. Yu, M. Maddah-Ali, and S. Avestimehr, "Polynomial codes: An optimal design for high-dimensional coded matrix multiplication," in *Proc. 30th Adv. Neural Inf. Process. Syst. (NIPS)*, Long Beach, CA, USA, 2017, pp. 4403–4413.
- [84] S. Dutta, M. Fahim, F. Haddadpour, H. Jeong, V. Cadambe, and P. Grover, "On the optimal recovery threshold of coded matrix multiplication," *IEEE Trans. Inf. Theory*, vol. 66, no. 1, pp. 278–301, Jan. 2020.
- [85] S. Wang, J. Liu, and N. Shroff, "Coded sparse matrix multiplication," 2018. [Online]. Available: arXiv:1802.03430.
- [86] R. Tandon, Q. Lei, A. G. Dimakis, and N. Karampatziakis, "Gradient coding: Avoiding stragglers in distributed learning," in *Proc. 34th Int. Conf. Mach. Learn.*, vol. 70, 2017, pp. 3368–3376.
- [87] N. Raviv, R. Tandon, A. Dimakis, and I. Tamo, "Gradient coding from cyclic MDS codes and expander graphs," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 4305–4313.
- [88] W. Halbawi, N. Azizan, F. Salehi, and B. Hassibi, "Improving distributed gradient descent using Reed-Solomon codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Vail, CO, USA, 2018, pp. 2027–2031.
- [89] S. Li, S. M. M. Kalan, A. S. Avestimehr, and M. Soltanolkotabi, "Near-optimal straggler mitigation for distributed gradient methods," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, Vancouver, BC, Canada, 2018, pp. 857–866.
- [90] S. Li, S. M. M. Kalan, Q. Yu, M. Soltanolkotabi, and A. S. Avestimehr, "Polynomially coded regression: Optimal straggler mitigation via data encoding," 2018. [Online]. Available: arXiv:1805.09934.
- [91] S. Dutta, V. Cadambe, and P. Grover, "Coded convolution for parallel and distributed computing within a deadline," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Aachen, Germany, 2017, pp. 2403–2407.
- [92] Q. Yu, M. A. Maddah-Ali, and A. S. Avestimehr, "Coded Fourier transform," in *Proc. 55th Annu. Allerton Conf. Commun. Control Comput. (Allerton)*, Monticello, IL, USA, 2017, pp. 494–501.
- [93] M. Blaum and R. M. Roth, "On lowest density MDS codes," *IEEE Trans. Inf. Theory*, vol. 45, no. 1, pp. 46–59, Jan. 1999.
- [94] S. Balakrishnama and A. Ganapathiraju, "Linear discriminant analysis—A brief tutorial," *Inst. Signal Inf. Process.*, vol. 18, pp. 1–8, Jan. 1998.
- [95] H. Abdi and L. J. Williams, "Principal component analysis," *Interdiscipl. Rev. Comput. Stat.*, vol. 2, no. 4, pp. 433–459, 2010.
- [96] A. Severinson, A. G. I. Amat, and E. Rosnes, "Block-diagonal and LT codes for distributed computing with straggling servers," *IEEE Trans. Commun.*, vol. 67, no. 3, pp. 1739–1753, Mar. 2019.
- [97] H. Park and J. Moon, "Irregular product coded computation for high-dimensional matrix multiplication," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Paris, France, 2019, pp. 1782–1786.
- [98] T. Baharav, K. Lee, O. Ocal, and K. Ramchandran, "Straggler-proofing massive-scale distributed matrix multiplication with D-dimensional product codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Vail, CO, USA, 2018, pp. 1993–1997.
- [99] Q. Yu, M. A. Maddah-Ali, and A. S. Avestimehr, "Straggler mitigation in distributed matrix multiplication: Fundamental limits and optimal coding," *IEEE Trans. Inf. Theory*, vol. 66, no. 3, pp. 1920–1933, Mar. 2020.
- [100] M. Fahim and V. R. Cadambe, "Numerically stable polynomially coded computing," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Paris, France, 2019, pp. 3017–3021.
- [101] S. Dutta, Z. Bai, H. Jeong, T. M. Low, and P. Grover, "A unified coded deep neural network training strategy based on generalized PolyDot codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Vail, CO, USA, 2018, pp. 1585–1589.
- [102] G. Suh, K. Lee, and C. Suh, "Matrix sparsification for coded matrix multiplication," in *Proc. 55th Annu. Allerton Conf. Commun. Control Comput. (Allerton)*, Monticello, IL, USA, 2017, pp. 1271–1278.
- [103] M. Ye and E. Abbe, "Communication-computation efficient gradient coding," 2018. [Online]. Available: arXiv:1802.03475.

- [104] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "A unified coding framework for distributed computing with straggling servers," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Washington, DC, USA, 2016, pp. 1–6.
- [105] J. Zhang and O. Simeone, "Improved latency-communication trade-off for map-shuffle-reduce systems with stragglers," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, Brighton, U.K., 2019, pp. 8172–8176.
- [106] E. Parrinello, E. Lampiris, and P. Elia, "Coded distributed computing with node cooperation substantially increases speedup factors," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Vail, CO, USA, 2018, pp. 1291–1295.
- [107] K. Konstantinidis and A. Ramamoorthy, "Leveraging coding techniques for speeding up distributed computing," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Abu Dhabi, UAE, 2018, pp. 1–6.
- [108] S. Prakash, A. Reiszadeh, R. Pedarsani, and S. Avestimehr, "Coded computing for distributed graph analytics," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Vail, CO, USA, 2018, pp. 1221–1225.
- [109] S. R. Srinivasavaradhan, L. Song, and C. Fragouli, "Distributed computing trade-offs with random connectivity," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Vail, CO, USA, 2018, pp. 1281–1285.
- [110] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "Coded distributed computing: Straggling servers and multistage dataflows," in *Proc. 54th Annu. Allerton Conf. Commun. Control Comput. (Allerton)*, Monticello, IL, USA, 2016, pp. 164–171.
- [111] N. Woolsey, R. Chen, and M. Ji, "Cascaded coded distributed computing on heterogeneous networks," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Paris, France, 2019, pp. 2644–2648.
- [112] M. Kiamari, C. Wang, and A. S. Avestimehr, "On heterogeneous coded distributed computing," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Singapore, 2017, pp. 1–7.
- [113] M. A. Attia and R. Tandon, "Information theoretic limits of data shuffling for distributed learning," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Washington, DC, USA, 2016, pp. 1–6.
- [114] S. Gupta and V. Lalitha, "Locality-aware hybrid coded MapReduce for server-rack architecture," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Kaohsiung, Taiwan, 2017, pp. 459–463.
- [115] D. Stinson, *Combinatorial Designs: Constructions and Analysis*. New York, NY, USA: Springer, 2007.
- [116] K. Konstantinidis and A. Ramamoorthy, "Resolvable designs for speeding up distributed computing," *IEEE/ACM Trans. Netw.*, vol. 28, no. 4, pp. 1657–1670, Aug. 2020.
- [117] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "Compressed Coded Distributed Computing," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Vail, CO, USA, 2018, pp. 2032–2036.
- [118] N. Woolsey, R. Chen, and M. Ji, "A new combinatorial design of coded distributed computing," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Vail, CO, USA, 2018, pp. 726–730.
- [119] J. Jiang and L. Qu, "Cascaded coded distributed computing schemes based on placement delivery arrays," *IEEE Access*, vol. 8, pp. 221385–221395, 2020.
- [120] Q. Yan, X. Tang, and Q. Chen, "Placement delivery array and its applications," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Guangzhou, China, 2018, pp. 1–5.
- [121] V. Ramkumar and P. V. Kumar, "Coded MapReduce schemes based on placement delivery array," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Paris, France, 2019, pp. 3087–3091.
- [122] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Proc. 30th Adv. Neural Inf. Process. Syst. (NIPS)*, Long Beach, CA, USA, 2017, pp. 1709–1720.
- [123] L. Song, C. Fragouli, and T. Zhao, "A pliable index coding approach to data shuffling," *IEEE Trans. Inf. Theory*, vol. 66, no. 3, pp. 1333–1353, Mar. 2020.
- [124] S. Brahma and C. Fragouli, "Pliable index coding," *IEEE Trans. Inf. Theory*, vol. 61, no. 11, pp. 6192–6203, Nov. 2015.
- [125] Z. Charles, D. Papailiopoulos, and J. Ellenberg, "Approximate gradient coding via sparse random graphs," 2017. [Online]. Available: arXiv:1711.06771.
- [126] F. Haddadpour, Y. Yang, V. Cadambe, and P. Grover, "Cross-iteration coded computing," in *Proc. 56th Annu. Allerton Conf. Commun. Control Comput. (Allerton)*, Monticello, IL, USA, 2018, pp. 196–203.
- [127] F. Haddadpour, Y. Yang, M. Chaudhari, V. R. Cadambe, and P. Grover, "Straggler-resilient and communication-efficient distributed iterative linear solver," 2018. [Online]. Available: arXiv:1806.06140.
- [128] Y. Zhang, J. C. Duchi, and M. J. Wainwright, "Communication-efficient algorithms for statistical optimization," *J. Mach. Learn. Res.*, vol. 14, no. 1, pp. 3321–3363, 2013.
- [129] M. Zinkevich, M. Weimer, L. Li, and A. J. Smola, "Parallelized stochastic gradient descent," in *Proc. 23rd Adv. Neural Inf. Process. Syst. (NIPS)*, Vancouver, BC, Canada, 2010, pp. 2595–2603.
- [130] K. Wan, M. Ji, and G. Caire, "Topological coded distributed computing," 2020. [Online]. Available: arXiv:2004.04421.
- [131] W. Xia, P. Zhao, Y. Wen, and H. Xie, "A survey on data center networking (DCN): Infrastructure and operations," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 1, pp. 640–656, 1st Quart., 2017.
- [132] M. Al-Fares, A. Loukissas, and A. Vahdat, "A scalable, commodity data center network architecture," in *Proc. ACM SIGCOMM Conf. Data Commun. (SIGCOMM)*, Seattle, WA, USA, 2008, pp. 63–74.
- [133] J. Chung, K. Lee, R. Pedarsani, D. Papailiopoulos, and K. Ramchandran, "UberShuffle: Communication-efficient Data Shuffling for SGD via Coding Theory," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 1–7.
- [134] N. Woolsey, R. Chen, and M. Ji, "Coded distributed computing with heterogeneous function assignments," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Dublin, Ireland, 2020, pp. 1–6.
- [135] L. Song, S. R. Srinivasavaradhan, and C. Fragouli, "The benefit of being flexible in distributed computation," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Kaohsiung, Taiwan, 2017, pp. 289–293.
- [136] F. Xu and M. Tao, "Heterogeneous coded distributed computing: Joint design of file allocation and function assignment," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Waikoloa, HI, USA, 2019, pp. 1–6.
- [137] Q. Yu, S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "How to optimally allocate resources for coded distributed computing?," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Paris, France, 2017, pp. 1–7.
- [138] M. Zhao, W. Wang, Y. Wang, and Z. Zhang, "Load scheduling for distributed edge computing: A communication-computation tradeoff," *Peer-to-Peer Netw. Appl.*, vol. 12, no. 5, pp. 1418–1432, 2019.
- [139] K. Lee, R. Pedarsani, D. Papailiopoulos, and K. Ramchandran, "Coded computation for multicore setups," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Aachen, Germany, 2017, pp. 2413–2417.
- [140] M. Zaharia, A. Konwinski, A. D. Joseph, R. H. Katz, and I. Stoica, "Improving MapReduce performance in heterogeneous environments," in *Proc. USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, San Diego, CA, USA, 2008, p. 7.
- [141] S. Dutta, H. Jeong, Y. Yang, V. Cadambe, T. M. Low, and P. Grover, "Addressing unreliability in emerging devices and Non-Von Neumann architectures using coded computing," *Proc. IEEE*, vol. 108, no. 8, pp. 1219–1234, Aug. 2020.
- [142] K. G. Narra, Z. Lin, M. Kiamari, S. Avestimehr, and M. Annavaram, "Slack squeeze coded computing for adaptive straggler mitigation," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal. (SC)*, 2019, pp. 1–16.
- [143] C.-S. Yang, R. Pedarsani, and A. S. Avestimehr, "Timely-throughput optimal coded computing over cloud networks," in *Proc. 20th ACM Int. Symp. Mobile Ad Hoc Netw. Comput. (Mobihoc)*, Catania, Italy, 2019, pp. 301–310.
- [144] E. Ozfatura, D. Gündüz, and S. Ulukus, "Speeding up distributed gradient descent by utilizing non-persistent stragglers," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Paris, France, 2019, pp. 2729–2733.
- [145] N. Ferdinand and S. C. Draper, "Hierarchical coded computation," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Vail, CO, USA, 2018, pp. 1620–1624.
- [146] A. Reiszadeh, S. Prakash, R. Pedarsani, and A. S. Avestimehr, "Coded computation over heterogeneous clusters," *IEEE Trans. Inf. Theory*, vol. 65, no. 7, pp. 4227–4242, Jul. 2019.
- [147] M. Kim, J. Sohn, and J. Moon, "Coded matrix multiplication on a group-based model," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Paris, France, 2019, pp. 722–726.
- [148] D. Kim, H. Park, and J. Choi, "Optimal load allocation for coded distributed computation in heterogeneous clusters," 2019. [Online]. Available: arXiv:1904.09496.
- [149] Y. Keshkarjahromi, Y. Xing, and H. Seferoglu, "Dynamic heterogeneity-aware coded cooperative computation at the edge," in *Proc. IEEE 26th Int. Conf. Netw. Protocols (ICNP)*, Cambridge, U.K., 2018, pp. 23–33.
- [150] C. Yang, R. Pedarsani, and A. S. Avestimehr, "Timely coded computing," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Paris, France, 2019, pp. 2798–2802.

- [151] N. S. Ferdinand and S. C. Draper, "Anytime coding for distributed computation," in *Proc. 54th Annu. Allerton Conf. Commun. Control Comput. (Allerton)*, Monticello, IL, USA, 2016, pp. 954–960.
- [152] B. Wang, J. Xie, K. Lu, Y. Wan, and S. Fu, "On batch-processing based coded computing for heterogeneous distributed computing systems," 2019. [Online]. Available: arXiv:1912.12559.
- [153] J. Zhu, Y. Pu, V. Gupta, C. Tomlin, and K. Ramchandran, "A sequential approximation framework for coded distributed optimization," in *Proc. 55th Annu. Allerton Conf. Commun. Control Comput. (Allerton)*, Monticello, IL, USA, 2017, pp. 1240–1247.
- [154] T. Jahani-Nezhad and M. A. Maddah-Ali, "CodedSketch: Coded distributed computation of approximated matrix multiplication," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Paris, France, 2019, pp. 2489–2493.
- [155] V. Gupta, S. Wang, T. Courtade, and K. Ramchandran, "OverSketch: Approximate matrix multiplication for the cloud," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Seattle, WA, USA, 2018, pp. 298–304.
- [156] Z. Charles and D. Papailiopoulos, "Gradient coding via the stochastic block model," 2018. [Online]. Available: arXiv:1805.10378.
- [157] R. Bitar, M. Wootters, and S. El Rouayheb, "Stochastic gradient coding for straggler mitigation in distributed learning," *IEEE J. Sel. Areas Inf. Theory*, vol. 1, no. 1, pp. 277–291, May 2020.
- [158] C. Karakus, Y. Sun, S. N. Diggavi, and W. Yin, "Redundancy techniques for straggler mitigation in distributed optimization and learning," *J. Mach. Learn. Res.*, vol. 20, no. 72, pp. 1–47, 2019.
- [159] C. Karakus, Y. Sun, and S. Diggavi, "Encoded distributed optimization," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Aachen, Germany, 2017, pp. 2890–2894.
- [160] J. Kosaian, K. V. Rashmi, and S. Venkataraman, "Learning a code: Machine learning for approximate non-linear coded computation," 2018. [Online]. Available: arXiv:1806.01259.
- [161] D. P. Woodruff, "Sketching as a tool for numerical linear algebra," 2014. [Online]. Available: arXiv:1411.4357.
- [162] S. Wang, "A practical guide to randomized matrix computations with MATLAB implementations," 2015. [Online]. Available: arXiv:1505.07570.
- [163] G. Cormode and M. Hadjieleftheriou, "Finding frequent items in data streams," *Proc. VLDB Endow.*, vol. 1, no. 2, pp. 1530–1541, 2008.
- [164] H. Wang, Z. Charles, and D. Papailiopoulos, "ErasureHead: Distributed gradient descent without delays using approximate gradient coding," 2019. [Online]. Available: arXiv:1901.09671.
- [165] W. Chang and R. Tandon, "Random sampling for distributed coded matrix multiplication," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, Brighton, U.K., 2019, pp. 8187–8191.
- [166] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014. [Online]. Available: arXiv:1409.1556.
- [167] R. Hadidi, J. Cao, M. S. Ryoo, and H. Kim, "Robustly executing DNNs in IoT systems using coded distributed computing," in *Proc. 56th Annu. Design Autom. Conf. (DAC)*, Las Vegas, NV, USA, 2019, pp. 1–2.
- [168] A. B. Das, L. Tang, and A. Ramamoorthy, "C3LES: Codes for coded computation that leverage stragglers," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Guangzhou, China, 2018, pp. 1–5.
- [169] E. Ozfatura, S. Ulukus, and D. Gündüz, "Distributed gradient descent with coded partial gradient computations," in *Proc. ICASSP IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, Brighton, U.K., 2019, pp. 3492–3496.
- [170] E. Ozfatura, S. Ulukus, and D. Gunduz, "Coded distributed computing with partial recovery," 2020. [Online]. Available: arXiv:2004.03877.
- [171] Y. Yang, P. Grover, and S. Kar, "Coded distributed computing for inverse problems," in *Proc. 30th Adv. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 709–719.
- [172] W. Y. B. Lim et al., "Hierarchical incentive mechanism design for federated machine learning in mobile networks," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9575–9588, Oct. 2020.
- [173] W. Y. B. Lim et al., "Federated learning in mobile edge networks: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 2031–2063, 3rd Quart., 2020.
- [174] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. ACM 41st Annu. ACM Symp. Theory Comput. (STOC)*, Bethesda, MD, USA, 2009, pp. 169–178.
- [175] Z. Brakerski and V. Vaikuntanathan, "Efficient fully homomorphic encryption from (standard) LWE," *Soc. Ind. Appl. Math. J. Comput.*, vol. 43, no. 2, pp. 831–871, 2014.
- [176] O. Goldreich, *Secure Multi-Party Computation*, vol. 78, 1998.
- [177] Y. Huang, D. Evans, J. Katz, and L. Malka, "Faster secure two-party computation using garbled circuits," in *Proc. USENIX Security Symp.*, vol. 201, San Francisco, CA, USA, 2011, pp. 331–335.
- [178] D. Bogdanov, S. Laur, and J. Willemson, "ShareMind: A framework for fast privacy-preserving computations," in *Proc. Eur. Symp. Res. Comput. Security (ESORICS)*, 2008, pp. 192–206.
- [179] W. Chang and R. Tandon, "On the capacity of secure distributed matrix multiplication," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Abu Dhabi, UAE, 2018, pp. 1–6.
- [180] M. Ben-Or, S. Goldwasser, and A. Wigderson, "Completeness theorems for non-cryptographic fault-tolerant distributed computation," in *Proc. ACM Providing Sound Found. Cryptography*, 2019, pp. 351–371.
- [181] M. Kim, H. Yang, and J. Lee, "Private coded computation for machine learning," 2018. [Online]. Available: arXiv:1807.01170.
- [182] H. A. Nodehi, S. R. H. Najarkolaei, and M. A. Maddah-Ali, "Entangled polynomial coding in limited-sharing multi-party computation," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Guangzhou, China, 2018, pp. 1–5.
- [183] Y. Chen, L. Su, and J. Xu, "Distributed statistical machine learning in adversarial settings: Byzantine gradient descent," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 1, no. 2, pp. 1–25, 2017.
- [184] L. Chen, H. Wang, Z. Charles, and D. Papailiopoulos, "DRACO: Byzantine-resilient distributed training via redundant gradients," 2018. [Online]. Available: arXiv:1803.09877.
- [185] Q. Yu and A. S. Avestimehr, "Harmonic coding: An optimal linear code for privacy-preserving gradient-type computation," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Paris, France, 2019, pp. 1102–1106.
- [186] J. So, B. Guler, A. S. Avestimehr, and P. Mohassel, "CodedPrivateML: A fast and privacy-preserving framework for distributed machine learning," 2019. [Online]. Available: arXiv:1902.00641.
- [187] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [188] R. Bitar, P. Parag, and S. El Rouayheb, "Minimizing latency for secure distributed computing," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Aachen, Germany, 2017, pp. 2900–2904.
- [189] M. A. Hearst, S. T. Dumais, E. Osuna, J. Platt, and B. Scholkopf, "Support vector machines," *IEEE Intell. Syst. Appl.*, vol. 13, no. 4, pp. 18–28, Jul./Aug. 1998.
- [190] R. Bitar, P. Parag, and S. El Rouayheb, "Minimizing latency for secure coded computing using secret sharing via staircase codes," *IEEE Trans. Commun.*, vol. 68, no. 8, pp. 4609–4619, Aug. 2019.
- [191] J. Kakar, S. Ebadifar, and A. Sezgin, "Rate-efficiency and straggler-robustness through partition in distributed two-sided secure matrix computation," 2018. [Online]. Available: arXiv:1810.13006.
- [192] H. Yang and J. Lee, "Secure distributed computing with straggling servers using polynomial codes," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 1, pp. 141–150, Jan. 2019.
- [193] Z. Jia and S. A. Jafar, "On the capacity of secure distributed matrix multiplication," 2019. [Online]. Available: arXiv:1908.06957.
- [194] R. G. L. D'Oliveira, S. E. Rouayheb, and D. Karpuk, "GASP codes for secure distributed matrix multiplication," *IEEE Trans. Inf. Theory*, vol. 66, no. 7, pp. 4038–4050, Jul. 2020.
- [195] G. L. R. D'Oliveira, S. E. Rouayheb, D. Heinlein, and D. Karpuk, "Degree tables for secure distributed matrix multiplication," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Visby, Sweden, 2019, pp. 1–5.
- [196] X. Chang, J. Wang, J. Wang, V. Lee, K. Lu, and Y. Yang, "On achieving maximum secure throughput using network coding against wiretap attack," in *Proc. IEEE 30th Int. Conf. Distrib. Comput. Syst.*, Genova, Italy, 2010, pp. 526–535.
- [197] R. Zhao et al., "Weakly secure coded distributed computing," in *Proc. IEEE SmartWorld Ubiquitous Intell. Comput. Adv. Trust. Comput. Scalable Comput. Commun. Cloud Big Data Comput. Internet People Smart City Innov. (SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI)*, Guangzhou, China, 2018, pp. 603–610.
- [198] P. Resnick, K. Kuwabara, R. Zeckhauser, and E. Friedman, "Reputation systems," *Commun. ACM*, vol. 43, no. 12, pp. 45–48, 2000.
- [199] R. Mijumbi, J. Serrat, J. Gorricho, N. Bouten, F. De Turck, and R. Boutaba, "Network function virtualization: State-of-the-art and research challenges," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 1, pp. 236–262, 1st Quart., 2016.
- [200] Q. Qi, W. Wang, X. Gong, and X. Que, "A SDN-based network virtualization architecture with autonomie management," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Austin, TX, USA, 2014, pp. 178–182.
- [201] V. Eramo, T. Catena, and F. G. Lavacca, "Proposal and investigation of an optical reconfiguration cost aware policy for resource allocation in network function virtualization infrastructures," in *Proc. 21st Int. Conf. Transp. Opt. Netw. (ICTON)*, Angers, France, 2019, pp. 1–5.

- [202] J. Sun *et al.*, "A reliability-aware approach for resource efficient virtual network function deployment," *IEEE Access*, vol. 6, pp. 18238–18250, 2018.
- [203] A. Al-Quzweeni, T. E. H. El-Gorashi, L. Nonde, and J. M. H. Elmirkhani, "Energy efficient network function virtualization in 5G networks," in *Proc. 17th Int. Conf. Transp. Opt. Netw. (ICTON)*, Budapest, Hungary, 2015, pp. 1–4.
- [204] L. Linguaglossa *et al.*, "Survey of performance acceleration techniques for network function virtualization," *Proc. IEEE*, vol. 107, no. 4, pp. 746–764, Apr. 2019.
- [205] H. Jang, J. Jeong, H. Kim, and J. Park, "A survey on interfaces to network security functions in network virtualization," in *Proc. IEEE 29th Int. Conf. Adv. Inf. Netw. Appl. Workshops*, Gwangju, South Korea, 2015, pp. 160–163.
- [206] A. Aljuhani and T. Alharbi, "Virtualized network functions security attacks and vulnerabilities," in *Proc. IEEE 7th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Las Vegas, NV, USA, 2017, pp. 1–4.
- [207] S. Cherrared, S. Imadali, E. Fabre, G. Gössler, and I. G. B. Yahia, "A survey of fault management in network virtualization environments: Challenges and solutions," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 4, pp. 1537–1551, Dec. 2019.
- [208] P. v. Anvith, N. Gunavathi, B. Malarkodi, and B. Rebekka, "A survey on network functions virtualization for telecom paradigm," in *Proc. TEQIP III Sponsored Int. Conf. Microw. Integr. Circuits Photon. Wireless Netw. (IMICPW)*, Tiruchirappalli, India, 2019, pp. 302–306.
- [209] J. Liu, Z. Jiang, N. Kato, O. Akashi, and A. Takahara, "Reliability evaluation for NFV deployment of future mobile broadband networks," *IEEE Wireless Commun.*, vol. 23, no. 3, pp. 90–96, Jun. 2016.
- [210] A. Al-Shuwaili, O. Simeone, J. Klierer, and P. Popovski, "Coded network function virtualization: Fault tolerance via in-network coding," *IEEE Wireless Commun. Lett.*, vol. 5, no. 6, pp. 644–647, Dec. 2016.
- [211] N. Nikaein, "Processing radio access network functions in the cloud: Critical issues and modeling," in *Proc. 6th Int. Workshop Mobile Cloud Comput. Services (MCS)*, Paris, France, 2015, pp. 36–43.
- [212] M. Aliasgari, J. Klierer, and O. Simeone, "Coded computation against processing delays for virtualized cloud-based channel decoding," *IEEE Trans. Commun.*, vol. 67, no. 1, pp. 28–38, Jan. 2019.
- [213] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," 2018. [Online]. Available: arXiv:1803.03635.
- [214] S. Li, Q. Yu, M. A. Maddah-Ali, and A. S. Avestimehr, "A scalable framework for wireless distributed computing," *IEEE/ACM Trans. Netw.*, vol. 25, no. 5, pp. 2643–2654, Oct. 2017.
- [215] M. Kiamari, C. Wang, and A. S. Avestimehr, "Coding for edge-facilitated wireless distributed computing with heterogeneous users," in *Proc. 51st Asilomar Conf. Signals Syst. Comput.*, 2017, pp. 536–540.
- [216] H. Chen and Y. Wu, "Coded computing for master-aided distributed computing systems," 2020. [Online]. Available: arXiv:2004.03877.
- [217] S. Dhakal, S. Prakash, Y. Yona, S. Talwar, and N. Himayat, "Coded computing for distributed machine learning in wireless edge network," in *Proc. IEEE 90th Veh. Technol. Conf. (VTC-Fall)*, Honolulu, HI, USA, 2019, pp. 1–6.
- [218] S. Dhakal, S. Prakash, Y. Yona, S. Talwar, and N. Himayat, "Coded federated learning," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Waikoloa, HI, USA, 2019, pp. 1–6.
- [219] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Stat. (AISTATS)*, Ft. Lauderdale, FL, USA, 2017, pp. 1273–1282.
- [220] S. Prakash, S. Dhakal, M. Akdeniz, A. S. Avestimehr, and N. Himayat, "Coded computing for federated learning at the edge," 2020. [Online]. Available: arXiv:2007.03273.
- [221] K. Yang, Y. Shi, and Z. Ding, "Data shuffling in wireless distributed computing via low-rank optimization," *IEEE Trans. Signal Process.*, vol. 67, no. 12, pp. 3087–3099, Jun. 2019.
- [222] N. Zhao, F. R. Yu, M. Jin, Q. Yan, and V. C. M. Leung, "Interference alignment and its applications: A survey, research issues, and challenges," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 3, pp. 1779–1803, 3rd Quart., 2016.
- [223] F. Li, J. Chen, and Z. Wang, "Wireless MapReduce distributed computing," *IEEE Trans. Inf. Theory*, vol. 65, no. 10, pp. 6101–6114, Oct. 2019.
- [224] S. Ha, J. Zhang, O. Simeone, and J. Kang, "Wireless Map-Reduce distributed computing with full-duplex radios and imperfect CSI," in *Proc. IEEE 20th Int. Workshop Signal Process. Adv. Wireless Commun. (SPAWC)*, Cannes, France, 2019, pp. 1–5.
- [225] S. Ha, J. Zhang, O. Simeone, and J. Kang, "Coded federated computing in wireless networks with straggling devices and imperfect CSI," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Paris, France, 2019, pp. 2649–2653.
- [226] B. Wang, J. Xie, K. Lu, Y. Wan, and S. Fu, "Coding for heterogeneous UAV-based networked airborne computing," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Waikoloa, HI, USA, 2019, pp. 1–6.
- [227] J. S. Ng *et al.*, "Joint auction-coalition formation framework for communication-efficient federated learning in UAV-enabled Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2326–2344, Apr. 2021.
- [228] W. Y. B. Lim *et al.*, "Towards federated learning in UAV-enabled Internet of Vehicles: A multi-dimensional contract-matching approach," 2020. [Online]. Available: arXiv:2004.03877.
- [229] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proc. 15th ACM Workshop Hot Topics Netw. (HotNets)*, Atlanta, GA, USA, 2016, pp. 50–56.
- [230] Z. Du, C. Wu, T. Yoshinaga, K. L. A. Yau, Y. Ji, and J. Li, "Federated learning for vehicular Internet of Things: Recent advances and open issues," *IEEE Open J. Comput. Soc.*, vol. 1, pp. 45–61, 2020.
- [231] N. Woolsey, R.-R. Chen, and M. Ji, "A combinatorial design for cascaded coded distributed computing on general networks," 2020. [Online]. Available: arXiv:2004.03877.
- [232] M. Dai, Z. Zheng, S. Zhang, H. Wang, and X. Lin, "SAZD: A low computational load coded distributed computing framework for IoT systems," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3640–3649, Apr. 2020.
- [233] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 450–465, Feb. 2018.
- [234] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st Ed. MCC Workshop Mobile Cloud Comput. (MCC)*, Helsinki, Finland, 2012, pp. 13–16.
- [235] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "Coding for distributed fog computing," *IEEE Commun. Mag.*, vol. 55, no. 4, pp. 34–40, Apr. 2017.
- [236] H. Park, K. Lee, J. Sohn, C. Suh, and J. Moon, "Hierarchical coding for distributed computing," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Vail, CO, USA, 2018, pp. 1630–1634.
- [237] N. Kim, D. Kim, J. Lee, D. Niyato, and J. K. Choi, "Incentive-based coded distributed computing management for latency reduction in IoT services—A game theoretic approach," *IEEE Internet Things J.*, vol. 8, no. 10, pp. 8259–8278, May 2021.
- [238] Y. Sun, J. Zhao, S. Zhou, and D. Gunduz, "Heterogeneous coded computation across heterogeneous workers," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Waikoloa, HI, USA, 2019, pp. 1–6.



Jer Shyuan Ng (Graduate Student Member, IEEE) received the double degree (Hons.) in electrical engineering (Highest Distinction) and economics from the National University of Singapore in 2019. She is currently pursuing the Ph.D. degree with Alibaba Group and Alibaba-NTU Joint Research Institute, Nanyang Technological University (NTU), Singapore. Her research interests include incentive mechanisms and edge computing.



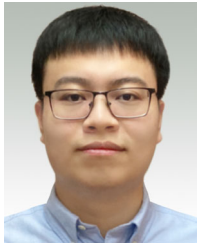
Wei Yang Bryan Lim received the double degrees (First Class Hons.) in economics and business administration (Finance) from the National University of Singapore in 2018. He is currently pursuing the Ph.D. degree with Alibaba Group and Alibaba-NTU Joint Research Institute, Nanyang Technological University, Singapore. His research interests include federated learning and edge intelligence.



Nguyen Cong Luong is currently a Lecturer with the Faculty of Computer Science, PHENIKAA University, Hanoi, Vietnam. He is also a Researcher with PHENIKAA Research and Technology Institute, Hanoi, Vietnam. His research interests include signal processing and resource management in wireless networks.

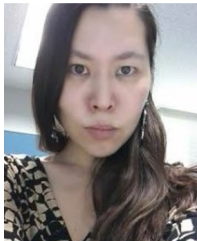


Cyril Leung (Life Member, IEEE) received the B.Sc. degree (First Class Hons.) from Imperial College, University of London, U.K., and the M.S. and Ph.D. degrees in electrical engineering from Stanford University. He has been an Assistant Professor with the Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology, and the Department of Systems Engineering and Computing Science, Carleton University. Since 1980, he has been with the Department of Electrical and Computer Engineering, The University of British Columbia (UBC), Vancouver, Canada, where he is a Professor and currently holds the PMC-Sierra Professorship in Networking and Communications. He served as Associate Dean of Research and Graduate Studies with the Faculty of Applied Science, UBC, from 2008 to 2011. His research interests include wireless communication systems, data security and technologies to support ageless aging for the elderly. He is a member of the Association of Professional Engineers and Geoscientists of British Columbia, Canada.



Zehui Xiong (Member, IEEE) received the Ph.D. degree from Nanyang Technological University, Singapore. He is currently an Assistant Professor with the Pillar of Information Systems Technology and Design, Singapore University of Technology and Design. Prior to that, he was a Researcher with Alibaba-NTU Joint Research Institute, Singapore. He was the Visiting Scholar with Princeton University and the University of Waterloo. He has published more than 100 research papers in leading journals and flagship conferences and four of them

are ESI Highly Cited Papers. His research interests include wireless communications, network games and economics, blockchain, and edge intelligence. He has won five best paper awards in international conferences and technical committee. He is a recipient of the Chinese Government Award for Outstanding Students Abroad in 2019, and the NTU SCSE Best Ph.D. Thesis Runner-Up Award in 2020. He is currently serving as the editor or guest editor for many leading journals including IEEE Transactions. He is the Founding Vice Chair of Special Interest Group on Wireless Blockchain Networks in IEEE Cognitive Networks Technical Committee.



Alia Asheralieva received the Ph.D. degree from the University of Newcastle, Australia, in 2014. From 2015 to 2016, she was with the Graduate School of Information Science and Technology, Hokkaido University, Japan. In 2017, she was with the Information Systems Technology and Design Pillar, Singapore University of Technology and Design. She is currently with the Department of Computer Science and Engineering, Southern University of Science and Technology, China. Her main research interests span many areas of communications and

networking, including cloud/edge/fog computing, blockchains, device-to-device and Internet of Things communications, game theory, machine learning, stochastic, and combinatorial optimization.



Dusit Niyato (Fellow, IEEE) received the B.Eng. degree from the King Mongkut's Institute of Technology Ladkrabang, Bangkok, Thailand, in 1999, and the Ph.D. degree in electrical and computer engineering from the University of Manitoba, Winnipeg, MB, Canada, in 2008. He is currently a Professor with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. His research interests are in the areas of Internet of Things, machine learning, and incentive mechanism design.



Chunyan Miao (Senior Member, IEEE) received the B.S. degree from Shandong University, Jinan, China, in 1988, and the M.S. and Ph.D. degrees from Nanyang Technological University, Singapore, in 1998 and 2003, respectively. She is currently a Professor with the School of Computer Science and Engineering, Nanyang Technological University, and the Director of the Joint NTU-UBC Research Centre of Excellence in Active Living for the Elderly (LILY). Her research focus on infusing intelligent agents into interactive new media (virtual, mixed, mobile, and pervasive media) to create novel experiences and dimensions in game design, interactive narrative, and other real world agent systems.